Granular-Ball Local-Consensus Soft Supervision for Noisy-Label Transductive Node Classification with Graph-Free Inference

Abstract

In many deployment scenarios, graph structure is available during training, whereas inference must rely only on node attributes because relations are missing or online graph processing is constrained. Methods for learning with noisy labels on graphs generally retain graph-dependent inference, while graph-free node classification has largely been studied with clean supervision. We propose Granular-Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification with noisy labels and graph-free inference. GB-LCSS first learns label-free node representations from attributes and graph connectivity and recursively partitions the resulting space into a bounded set of disjoint granular balls. For each training node, GB-LCSS aggregates the labels of the other training nodes in the same ball and adaptively blends their consensus with the node's observed label to form a soft target. Label-free region construction prevents noisy annotations from shaping the partition, while the adaptive target reduces the influence of isolated label errors when local evidence is consistent. Across five datasets and five noise settings, GB-LCSS achieves the highest or tied-highest five-seed mean accuracy in 24 of 25 settings among seven noisy-label baselines using the same deployed MLP architecture, while requiring only node attributes at inference.

Keywords: label noise, node classification, graph-free inference, graph contrastive learning, granular-ball computing, knowledge distillation

1. Introduction

Graph neural networks (GNNs) are effective for node classification because message passing combines node attributes with relational context [1, 2]. This dependency also complicates deployment: prediction requires access to the graph and repeated neighborhood aggregation, which may be costly or infeasible when relations are unavailable or online graph processing is restricted. Graph-free approaches such as Graph-MLP and GLNN exploit graph structure during training but deploy an MLP that predicts directly from node attributes [3, 4]. This removes graph access and neighborhood aggregation from online prediction.

Noisy training labels introduce a second challenge. Labels collected through manual annotation, automated rules, or multiple sources can contain errors, and deep models may eventually memorize corrupted supervision [8]. On graphs, message passing can further spread the effect of mislabeled nodes [13]. Noise-robust graph methods use topology for label correction, pseudo-labeling, sample selection, or robust representation learning [9–12, 47], but their predictions generally remain graph dependent. Existing graph-free studies, by contrast, have largely focused on structural knowledge transfer with clean supervision. Robustness to noisy training labels remains insufficiently studied in graph-free node classification.

To improve supervision without introducing graph dependence at inference, GB-LCSS uses the training graph to construct soft targets for the deployed MLP. The method first learns label-free node representations with a graph contrastive encoder instantiated by GRACE [16], so label noise does not alter the geometry used for partitioning. It then recursively divides this representation space into a bounded set of disjoint granular balls [34]. For each training node, GB-LCSS counts the observed labels of the other training nodes in the same ball to form a local consensus. Greater agreement among these labels gives the consensus more weight, helping counter isolated label errors, whereas

weak agreement retains more of the node's observed label. The resulting node-specific soft targets train an MLP that receives only node attributes at inference.

We evaluate GB-LCSS on Cora, CiteSeer, CoraFull, Coauthor-CS, and Amazon-Photo under uniform, pair, and instance-dependent label noise. Across five noise settings, it achieves the highest or tied-highest five-seed mean accuracy in 24 of 25 settings among seven noise-robust baselines using the same deployed MLP architecture. A secondary graph-teacher extension trains on the same consensus targets and supplies predictions over all nodes, improving the MLP student's mean accuracy without changing its inference inputs.

The main contributions are as follows:

1. We study noisy-label transductive node classification with graph-free inference, using training-time graph structure to improve supervision while the deployed classifier uses only node attributes at inference.
2. We introduce GB-LCSS, which partitions a label-free graph contrastive representation into disjoint granular balls and uses label agreement among the other training nodes in each ball to refine each node's observed label into a soft target.
3. We evaluate GB-LCSS against noise-robust and structure-assisted graph-free methods across five datasets and five noise settings. Controlled grouping and target analyses identify the contribution of local consensus, and a secondary graph-teacher extension examines whether node-wise predictions over all nodes can further improve the MLP student.

2. Related work

2.1. Learning with noisy labels

Learning with noisy labels reduces the influence of corrupted annotations by controlling which samples are trusted, how supervision is represented, or how strongly a model fits uncertain targets. Sample-selection methods exploit the tendency of deep networks to fit clean patterns before memorizing noise. Co-teaching exchanges small-loss samples between two networks, while JoCoR also enforces agreement between their predictions [17, 18]. DivideMix treats the separated clean and noisy samples as a semi-supervised learning problem, whereas FINE and CT estimate sample reliability from class-wise feature directions and confidence trajectories [23–25].

A complementary family modifies the training objective. SCE combines cross-entropy with reverse cross-entropy, ELR suppresses late-stage memorization, and ANL and LogitClip reduce sensitivity to corrupted targets through negative learning and logit-norm control [19–22]. These methods provide general strategies for learning with noisy labels but do not explicitly model graph relations between samples.

2.2. Noise-robust learning on graphs

Graph learning under label noise introduces an additional effect: message passing can propagate erroneous supervision through connected nodes. One line of work therefore uses graph structure as evidence for estimating label reliability. UnionNET, NRGNN, and RTGNN derive neighborhood evidence, augment reliable supervision, or coordinate dual networks [9, 10, 46]. Topology-guided selection and disagreement methods, including TSS, TFR, ProCon, GD^2, and GNN-CFGD, identify unreliable nodes from reconstruction, consistency, predictive discrepancy, or training history [40–42, 44, 47].

A second line of work uses graph structure to generate or propagate more reliable supervision. DND-NET constructs robust pseudo-labels from neighborhood uncertainty, while RNCGLN combines graph contrastive representation learning with label self-correction under label and structure perturbations [11, 12]. Prototype-Guided Supervision aggregates noisy labels into class-level prototypes, graph-teacher methods such as BO-NNC and SAMTD improve the source of supervision, and GTA, MILAB, and HALF adjust topology or aggregation to reduce the influence of unreliable neighborhoods [45, 50, 51, 58, 59, 63]. NoisyGL [13] provides a unified benchmark for this family of methods.

These methods establish graph structure as useful evidence for identifying unreliable labels and refining supervision. Their inference pipelines, however, generally continue to depend on the graph, leaving the graph-free deployment problem unresolved.

2.3. Graph-free node classification

Graph-free methods avoid graph-dependent online prediction by exploiting graph structure during training. GraphMLP incorporates adjacency through a neighborhood-contrastive objective, whereas GLNN distills predictions from a GNN teacher into an MLP; both deploy an attribute-only MLP [3, 4]. Other methods transfer graph information through self-distillation, representation alignment, or precomputed structural signals [5, 14, 15, 27, 33]. They mainly differ in how graph information is transferred during training and whether graph-derived inputs are retained at inference.

Teacher-based methods broaden the knowledge transferred to an MLP. FF-G2M separates low- and high-frequency components, KRD selects reliable teacher knowledge according to prediction stability, and VQGraph represents local structures with discrete codes [6, 7, 26]. PGKD and TINED transfer class prototypes or combine parameter injection with Dirichlet-energy distillation [28, 43], while AdaGMLP, ProGMLP, and SALE-MLP study adaptive transfer, accuracy–cost trade-offs, and structure-aware latent representations [29–31]. Yu et al. [32] theoretically explain how GNN soft labels convey graph-induced low-frequency bias to an MLP.

Graph-based label smoothing is closely related to GB-LCSS because it also modifies the training targets. PosteL estimates posterior soft labels from neighboring labels and graph-wide label co-occurrence statistics, ALS adjusts label-smoothing strength during graph training, and SALS incorporates neighborhood information into the target [60–62]. GB-LCSS also modifies the training targets, but defines its voting groups by geometry in a frozen, label-free graph representation rather than directly by graph adjacency. Each training node combines its observed label with the label consensus of the other training nodes in the same granular ball.

2.4. Graph contrastive learning and granular-ball learning

Graph contrastive learning provides graph-aware representations without node labels by enforcing consistency between augmented graph views. GRACE constructs stochastic views through edge dropping and feature masking and aligns the representations of corresponding nodes across those views [16]. In GB-LCSS, this representation-learning stage is instantiated with GRACE and completed before labels are aggregated. The resulting geometry captures graph context while keeping regional partitioning independent of noisy labels.

Granular-ball computing represents samples with coarse-grained hyperspherical regions for multi-granularity learning [34]. GBkNN classifies queries over granular balls, while granular-ball sampling uses ball purity and boundary structure to select representative samples [34, 37]. Noise-oriented extensions employ granular regions for coarse representation and classification, reliable-sample selection, pseudo-label construction, or adaptive error correction [35, 36, 52, 53]. In deep learning, GBRAIN, G-GBC, and BNO-GB use granular regions as coarse training units or as regions for distinguishing different forms of noisy samples [48, 49, 57]. Granular balls have also served as supernodes or hierarchical regions in supervised graph coarsening, GBGC, and MGGR [54–56].

Prior studies mainly use granular balls as classifiers, sample-selection structures, coarse training units, pseudo-label regions, or graph-coarsening units. GB-LCSS instead uses them as a regional interface for supervision construction: the balls are formed without labels, their statistics are reused to build node-specific targets, and classification remains with an MLP operating on individual node attributes. Within this design, graph contrastive learning supplies the geometry and granular balls organize the local evidence.

3. Method

3.1. Problem setting and notation

Consider a transductive attributed graph $G = (\mathcal{V}, \mathcal{E}, X)$, where $\mathcal{V} = \{v_1, \ldots, v_N\}$ is the node set and $\mathcal{E}$ is the edge set. Let $A \in \{0,1\}^{N\times N}$ be the adjacency matrix and $X = [x_1, \ldots, x_N]^\top \in \mathbb{R}^{N\times d}$ the node-attribute matrix. Each node

belongs to one of C classes. Let $\mathcal{V}_L$ denote the set of training nodes. The observed label $\widetilde{y}_v \in \{1, \ldots, C\}$ of a training node may be corrupted by an unknown noise process. GB-LCSS does not require prior knowledge of the noise type, noise rate, or label transition matrix.

We consider transductive graph-free inference on a fixed graph. During training, the full node-attribute matrix X and adjacency matrix A, including the test nodes, are available for representation learning and target construction, while test labels remain hidden. Only the classifier f_θ is retained after training. For a node v, its predictive distribution is

$$p_\theta(\cdot \mid x_v) = \mathrm{softmax}\left(f_\theta(x_v)\right), \tag{1}$$

where x_v is the classifier's only input at inference; the adjacency matrix A, neighboring nodes, the frozen representation, and the graph encoder are unavailable to the classifier.

Let $\mathcal{B} = \{B_j\}_{j=1}^{K}$ be a granular-ball partition covering all nodes, and let $T_j = B_j \cap \mathcal{V}_L$ be the training nodes in ball B_j. For $v \in T_j$, the subscript $-v$ indicates that the label of node v is removed when local consensus is constructed. Other symbols are defined at first use.

3.2. Overview of GB-LCSS

Figure 1 summarizes the four stages of GB-LCSS. First, the method learns a label-free node representation from attributes and graph connectivity, providing geometry for regional partitioning without using noisy labels. Second, recursive 2-means splitting partitions the frozen representation into a bounded set of disjoint granular balls. Third, GB-LCSS computes one label histogram per ball and subtracts each training node's own vote to construct its local consensus. Agreement among the remaining labels determines how much weight this consensus receives relative to the node's observed label. Finally, these node-specific targets train an MLP, and only this MLP is retained for graph-free inference.

3.3. Label-agnostic granular-ball construction

3.3.1. Graph contrastive representation

To construct local regions that reflect node attributes and graph connectivity without using noisy labels, we instantiate the representation-learning component with GRACE [16]. The component forms two stochastic graph views through edge dropping and feature masking, then aligns the representations of the same node across views. Pretraining uses X and A but no node labels. After pretraining, we use the encoder output before the GRACE projection head as the representation

$$H = g_\psi(X, A) \in \mathbb{R}^{N \times d_h}, \tag{2}$$

which we store in node order after freezing ψ. Before constructing the balls, each node representation is L2-normalized with $\varepsilon = 10^{-12}$ as a numerical safeguard:

$$\bar{h}_v = \frac{h_v}{\max(\|h_v\|_2, \varepsilon)}. \tag{3}$$

The normalized representation matrix is denoted by $\bar{H} = [\bar{h}_1, \ldots, \bar{h}_N]^\top$. Throughout the main experiments, the frozen GRACE representation defines the label-free geometry used for granular-ball construction. The partition is determined entirely by this geometry. Training labels are introduced only afterward to construct local-consensus targets. Appendix D.3 examines alternative representation sources and variation across GRACE pretraining seeds.

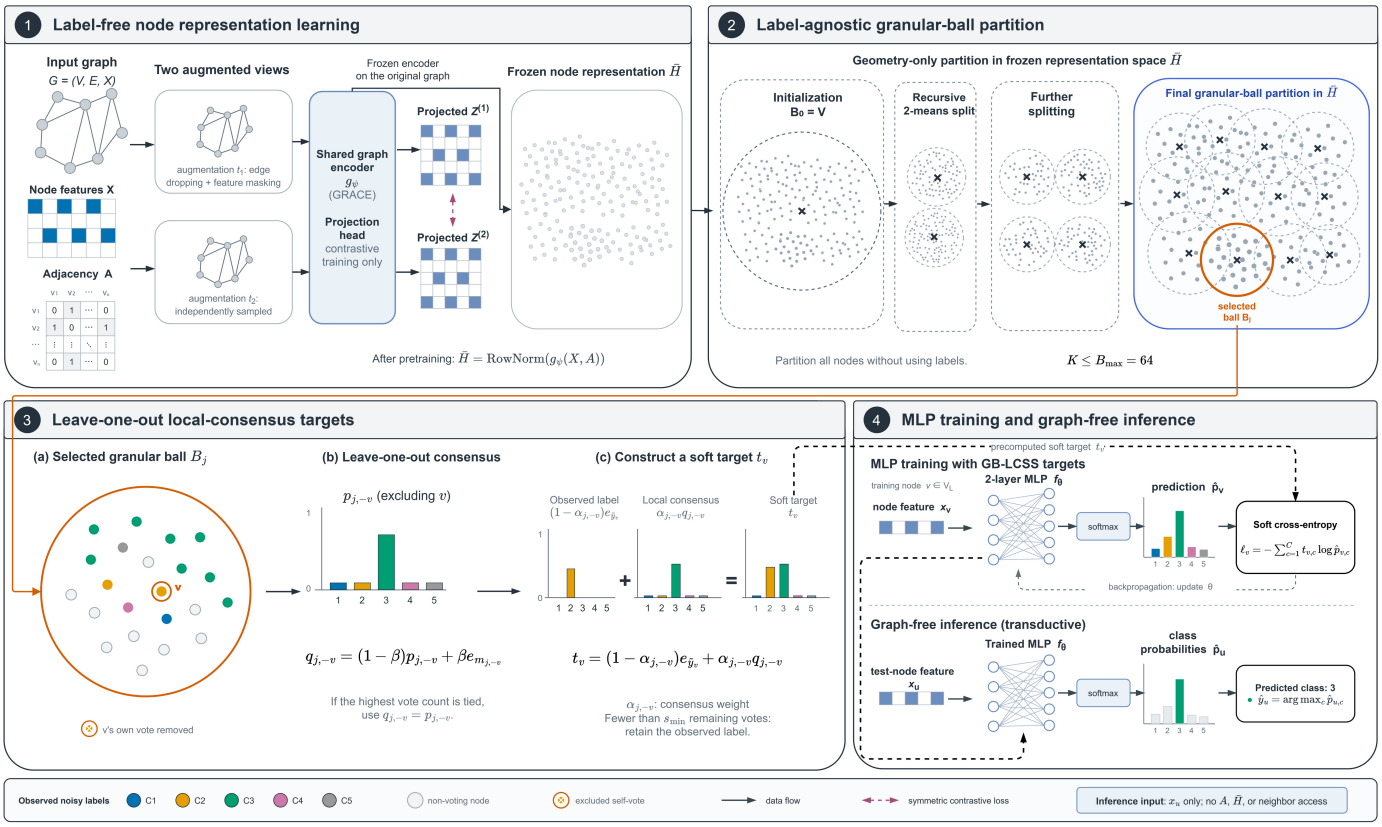


Figure 1. Overview of GB-LCSS. Stage 1 learns a label-free representation, and Stage 2 constructs granular balls in that representation space. Stage 3 enlarges a representative ball to show how a leave-one-out local-consensus target is formed. Stage 4 trains the deployed MLP with the resulting soft targets. Hollow nodes do not vote. Ball outlines are schematic, and each node belongs to exactly one ball. Graph structure and the learned representation are used only during training, whereas the classifier receives only node attributes at inference.

3.3.2. Granular-ball construction

The partitioning stage converts the frozen representation into a bounded set of compact, disjoint regions. Starting from one ball containing all nodes, GB-LCSS considers the most dispersed current regions first and accepts a split only when it sufficiently reduces within-region dispersion. The granular-ball set is initialized as $\mathcal{B} = \{\mathcal{V}\}$. For a ball $B \subseteq \mathcal{V}$, its center and mean distance to the center are defined as

$$\mu_B = \frac{1}{|B|} \sum_{v \in B} \bar{h}_v, \qquad D(B) = \frac{1}{|B|} \sum_{v \in B} \left\| \bar{h}_v - \mu_B \right\|_2 . \tag{4}$$

Each granular ball is represented by its member set and center, while $D(B)$ is its mean radial dispersion for evaluating candidate splits. A smaller $D(B)$ indicates a more compact region in the representation space. At each iteration, we collect candidate balls satisfying $|B| \ge 2n_{\min}$ and attempt 2-means splits in descending order of $D(B)$. The finite split budget is therefore allocated first to geometrically diffuse regions. If a candidate ball B is partitioned into $B^{(1)}$ and $B^{(2)}$, the size-weighted dispersion after splitting is

$$D_{\text{split}}(B) = \frac{|B^{(1)}| D(B^{(1)}) + |B^{(2)}| D(B^{(2)})}{|B|}. \tag{5}$$

In each iteration, we accept the first candidate split for which both child balls contain at least $n_{\min}$ nodes and

$$D_{\text{split}}(B) \le (1-\delta)D(B). \tag{6}$$

Here, $n_{\min}$ is a positive integer and $0 < \delta < 1$. Equation (6) requires the weighted dispersion to decrease by at least a fraction δ, while the child-size constraint prevents very small regions. After each accepted split, the candidates and their ordering are recomputed. Recursion stops when an iteration contains no valid split or the number of balls reaches the prescribed ball budget $B_{\max}$, where $B_{\max}$ is a positive integer. The final partition contains at most $B_{\max}$ disjoint granular balls, covers all nodes, and assigns each node to exactly one ball. Section 4.4 gives the experimental parameters.

3.4. Granular-ball local-consensus soft supervision

3.4.1. Leave-one-out consensus target

After the label-free partition has been fixed, GB-LCSS uses the observed labels of the training nodes within each ball to construct local supervision. One label histogram is computed per ball and reused across its training nodes. For each training node v, the node-specific leave-one-out counts are obtained by subtracting v's own vote.

For a granular ball B_j, let $T_j = B_j \cap \mathcal{V}_L$ denote its training nodes. To prevent a node's observed label from contributing to the local evidence used to adjust that same label, GB-LCSS excludes the node from the within-ball vote and defines the voting set for each $v \in T_j$ as

$$T_{j,-v} = T_j \backslash \{v\}. \tag{7}$$

When $|T_{j,-v}| \ge s_{\min}$, the noisy labels of the other training nodes in the ball define the empirical label distribution

$$p_{j,-v,c} = \frac{1}{|T_{j,-v}|} \sum_{u \in T_{j,-v}} \mathbb{I}(\widetilde{y}_u = c), \qquad c = 1, \dots, C. \tag{8}$$

Let

$$M_{j,-v} = \{c : p_{j,-v,c} = r_{j,-v}\}, \qquad r_{j,-v} = \max_c p_{j,-v,c}. \tag{9}$$

$M_{j,-v}$ is the set of classes tied for the largest number of votes after node v has been removed. Ties are determined directly from integer vote counts. If the most-voted class is unique, denote it by $m_{j,-v}$ and mix its one-hot vector with the empirical distribution. If multiple classes are tied, retain the empirical distribution:

$$q_{j,-v} = \begin{cases} (1-\beta)p_{j,-v} + \beta e_{m_{j,-v}}, & |M_{j,-v}| = 1, \\ p_{j,-v}, & |M_{j,-v}| > 1. \end{cases} \tag{10}$$

Here, e_c is the one-hot vector for class c, and $\beta \in [0,1]$ controls the mixture of the empirical distribution and the one-hot vector of the most-voted class. Under a tie, no class is selected by its index. Because a different node is removed each time, nodes in the same granular ball may have different $p_{j,-v}$ and $q_{j,-v}$.

3.4.2. Normalized consensus weight and node target

The weight controls how strongly local consensus changes the observed label. We assign more weight to the consensus when a larger proportion of the remaining labels agree on one class. We therefore normalize the largest vote proportion against the uniform class probability $1/C$:

$$\alpha_{j,-v} = \max\left(0, \frac{r_{j,-v} - 1/C}{1 - 1/C}\right). \tag{11}$$

$\alpha_{j,-v}$ measures the concentration of the leave-one-out label distribution relative to a uniform distribution, whereas the positive integer $s_{\min}$ specifies the minimum number of leave-one-out votes required to use local consensus. The two quantities therefore describe distributional concentration and effective vote count, respectively. For $v \in T_j$, the final training target is

$$t_v = \begin{cases} (1-\alpha_{j,-v})e_{\widetilde{y}_v} + \alpha_{j,-v} q_{j,-v}, & |T_{j,-v}| \geq s_{\min}, \\ e_{\widetilde{y}_v}, & |T_{j,-v}| < s_{\min}. \end{cases} \tag{12}$$

When the leave-one-out vote is close to uniform, $\alpha_{j,-v}$ is small and the target remains close to the node's observed label. As the label distribution becomes more concentrated, the supervision shifts toward the local consensus $q_{j,-v}$. The second branch retains the observed label when too few votes remain. If at least $s_{\min}$ votes remain and all agree, the target becomes the one-hot label of their shared class.

Equations (7)–(12) use only the observed labels of training nodes; validation and test labels do not enter target construction or the training loss.

3.4.3. Graph-free MLP training

The student f_θ is a two-layer MLP. Its hidden linear layer is followed by ReLU, batch normalization, and dropout, and its output layer produces the class logits. Its predictive distribution is given by Eq. (1), and it minimizes the soft cross-entropy over the training nodes:

$$\mathcal{L}_{\mathrm{LCSS}} = -\frac{1}{|\mathcal{V}_L|} \sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} t_{vc} \log p_\theta(c \mid x_v). \tag{13}$$

Algorithm 1: Recursive granular-ball partitioning in the frozen representation space

Input: Normalized frozen representation $\bar{H}$, node set $\mathcal{V}$, parameters $B_{\max}, n_{\min}, \delta$
Output: Set of disjoint granular balls $\mathcal{B}$

$\mathcal{B} \leftarrow \{\mathcal{V}\}$
while $|\mathcal{B}| < B_{\max}$ **do**
  $\mathcal{B}_{\mathrm{cand}} \leftarrow \{B \in \mathcal{B} : |B| \geq 2n_{\min}\}$, sorted by $D(B)$ in descending order
  accepted $\leftarrow$ false
  foreach $B \in \mathcal{B}_{\mathrm{cand}}$ **do**
    Partition B into $B^{(1)}$ and $B^{(2)}$ using 2-means
    if $\min(|B^{(1)}|, |B^{(2)}|) \geq n_{\min}$ *and* $D_{\mathrm{split}}(B) \leq (1-\delta)D(B)$ **then**
      $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{B\}) \cup \{B^{(1)}, B^{(2)}\}$
      accepted $\leftarrow$ true
      break
  if accepted = false **then**
    break
return $\mathcal{B}$

3.5. Consensus-teacher distillation extension

Basic GB-LCSS provides direct consensus targets only for training nodes. To extend this supervision across the complete graph, the distillation variant trains a graph teacher on the same targets and then trains the MLP student to match the teacher's node-wise predictions for all nodes. The teacher therefore serves as a graph-based intermediary during training, while the deployed model remains the same MLP.

Algorithm 2: Leave-one-out granular-ball soft-target construction

Input: Granular-ball set $\mathcal{B}$, noisy labels $\{\tilde{y}_v \mid v \in \mathcal{V}_L\}$, number of classes C, parameters $s_{\min}, \beta$
Output: Soft targets for training nodes $\{t_v \mid v \in \mathcal{V}_L\}$

Initialize $t_v \leftarrow e_{\tilde{y}_v}$ for every $v \in \mathcal{V}_L$
foreach $B_j \in \mathcal{B}$ **do**
  $T_j \leftarrow B_j \cap \mathcal{V}_L$
  $n_j \leftarrow \sum_{u \in T_j} e_{\tilde{y}_u}$
  foreach $v \in T_j$ **do**
    $n_{j,-v} \leftarrow n_j - e_{\tilde{y}_v}$
    if $\sum_c n_{j,-v,c} < s_{\min}$ **then**
      continue
    $p_{j,-v} \leftarrow n_{j,-v} / \sum_c n_{j,-v,c}$
    $r_{j,-v} \leftarrow \max_c p_{j,-v,c}$
    $M_{j,-v} \leftarrow \{c : p_{j,-v,c} = r_{j,-v}\}$
    if $|M_{j,-v}| = 1$ **then**
      $q_{j,-v} \leftarrow (1-\beta)p_{j,-v} + \beta e_m$, where $m \in M_{j,-v}$
    else
      $q_{j,-v} \leftarrow p_{j,-v}$
    $\alpha_{j,-v} \leftarrow \max\left(0, \frac{r_{j,-v} - 1/C}{1 - 1/C}\right)$
    $t_v \leftarrow (1-\alpha_{j,-v})e_{\tilde{y}_v} + \alpha_{j,-v} q_{j,-v}$
return $\{t_v \mid v \in \mathcal{V}_L\}$

The teacher is trained directly with the GB-LCSS targets defined in Section 3.4. For each training node $v \in T_j$, target construction follows the same leave-one-out vote and low-support fallback as basic GB-LCSS. The teacher target is

$$t_v^T = t_v, v \in \mathcal{V}_L. \tag{14}$$

Here, t_v is defined in Eq. (12). In the extension, this target supervises a graph-structured teacher. The teacher g_ϕ is a two-layer GCN that takes the original node attributes X and adjacency matrix A as input:

$$Z^T = g_\phi(X, A). \tag{15}$$

The frozen representation $\bar{H}$ determines the granular balls and their consensus targets but is not used as a teacher input. The teacher minimizes the following objective over training nodes:

$$\mathcal{L}_T = -\frac{1}{|\mathcal{V}_L|} \sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} t_{vc}^T \log \operatorname{softmax}\left(Z_v^T\right)_c. \tag{16}$$

After selecting a teacher checkpoint, we freeze g_ϕ and compute temperature-scaled soft predictions for all nodes, where $\tau > 0$ is the distillation temperature:

$$\pi_v = \operatorname{softmax}\left(\frac{Z_v^T}{\tau}\right). \tag{17}$$

The MLP student then fits these predictions on all N nodes:

$$\mathcal{L}_{\mathrm{KD}} = -\frac{\tau^2}{N} \sum_{v \in \mathcal{V}} \sum_{c=1}^{C} \pi_{vc} \log \operatorname{softmax}\left(\frac{f_\theta(x_v)}{\tau}\right)_c. \tag{18}$$

The basic method applies local-consensus targets only to $\mathcal{V}_L$, whereas Eq. (18) fits teacher predictions on every

node. The student loss consists solely of cross-entropy against the teacher's soft predictions. After training, only the student MLP is retained; the teacher, adjacency matrix, and frozen representation are not used for inference.

3.6. Computational complexity and inference resources

Before training the MLP, basic GB-LCSS performs graph contrastive pretraining followed by granular-ball construction and soft-target generation. The frozen representation requires $O(Nd_h)$ storage and can be reused across noise settings and downstream seeds on the same dataset. Because the cost of GRACE depends on the encoder configuration and pretraining schedule, Section 5.5 reports its measured runtime and peak GPU memory; the analysis below focuses on post-pretraining ball construction and target generation.

On the frozen representation, running 2-means for I iterations on a node subset S costs $O(I|S|d_h)$. Candidate balls processed within the same iteration are disjoint, so their total number of nodes is at most N. Because at most $B_{\max}-1$ splits can be accepted, a conservative upper bound for granular-ball construction is

$$O(IB_{\max}Nd_h). \tag{19}$$

For fixed I and $B_{\max}$, Eq. (19) is linear in the number of nodes N and the representation dimension d_h. During construction, the frozen representation requires $O(Nd_h)$ space, while node assignments, ball centers, and class histograms add $O(N + B_{\max}(d_h + C))$. Once the ball-level histograms are available, all training targets can be generated in $O(|\mathcal{V}_L|C)$ time by subtracting each node's vote from its ball histogram. The current implementation stores the targets in an $N \times C$ matrix, adding $O(NC)$ space.

After target construction, student optimization is the same as for the standard two-layer MLP. Let F denote the hidden width. With full-batch training, each epoch evaluates the MLP for all N nodes and applies the loss on the training-node mask; over T_S epochs, this costs $O(T_S N(dF + FC))$. The per-node inference cost is

$$O(dF + FC), \tag{20}$$

At deployment, only the student parameters, of order $O(dF + FC)$, are retained. The adjacency matrix, frozen GRACE representation, and granular-ball partition are not required, so the per-node inference cost in Eq. (20) is independent of the number of edges $|\mathcal{E}|$. The distillation extension adds the training cost of a two-layer GCN teacher, which is also removed after training. Section 5.5 further reports measured GRACE pretraining time and memory together with grouping-construction time and storage.

4. Experimental setup

4.1. Datasets and evaluation scope

Cora, CiteSeer, CoraFull, Coauthor-CS, and Amazon-Photo were evaluated under the full noise spectrum: uniform 0.2/0.4/0.6, pair 0.4, and instance-dependent 0.4. Six additional datasets were evaluated under uniform 0.4, and Appendix H reports the supplementary results.

Cora and CiteSeer were taken from the standard citation-network benchmarks [38], and the remaining datasets are public benchmarks commonly used in graph learning [39]. Table 1 summarizes the five full-spectrum datasets. For reporting Table 1, self-loops were removed and reciprocal directed entries were counted as one undirected edge. Edge homophily is the proportion of these undirected edges whose endpoints share the same class and is reported only as a graph characteristic.

Table 1. Dataset statistics. Edges are unique undirected pairs after self-loop removal, and the number of training nodes is based on the 20% split.

Dataset	Nodes	Undirected edges	Features	Classes	Training nodes	Edge homophily
Cora	2,708	5,278	1,433	7	541	0.810
CiteSeer	3,327	4,552	3,703	6	665	0.736
CoraFull	19,793	63,421	8,710	70	3,958	0.567
Coauthor-CS	18,333	81,894	6,805	15	3,666	0.808
Amazon-Photo	7,650	119,081	745	8	1,530	0.827

4.2. Data splits and label noise

We used a transductive node-classification protocol on a fixed graph. The node attributes and adjacency of the complete graph were available during training, so the attributes and topology of test nodes provided unlabeled context for GRACE pretraining and granular-ball partitioning. For seeds 0–4, nodes were randomly sampled without class stratification using the NoisyGL percentage-split routine, with target sizes $\lfloor 0.2N \rfloor$, $\lfloor 0.1N \rfloor$, and $\lfloor 0.7N \rfloor$ for training, validation, and testing, respectively. All methods shared the same split for a given dataset and seed. Under the default protocol, noise was injected only into training labels. Clean validation labels were used for checkpoint selection, and test labels were used only for final evaluation. Test-node classification accuracy was the primary evaluation metric. Appendix C additionally examines checkpoint selection when validation labels are corrupted.

Uniform noise satisfies $P_{ii} = 1 - \rho$ and $P_{ij} = \rho/(C-1)$, with $\rho \in \{0.2, 0.4, 0.6\}$. Pair noise uses $\rho = 0.4$ and flips class i to $((i-2) \bmod C) + 1$ with probability ρ, leaving it unchanged with probability $1 - \rho$. Instance-dependent noise was generated with the NoisyGL routine at a configured mean flip rate of 0.4 and norm_std = 0.1. For each dataset and seed, all methods shared the same noisy training-label realization.

4.3. Compared methods and comparison groups

The first group consists of noisy-label methods implemented under a common two-layer MLP backbone: MLP+CE, Small-loss, Co-teaching, JoCoR, SCE, ELR, LogitClip, and GB-LCSS. All methods use the same labeled training nodes and receive only node attributes at inference. Small-loss ranks samples by per-sample cross-entropy and retains the small-loss subset according to the forget-rate schedule. GB-LCSS uses training-time graph information to construct supervision under our problem setting. Co-teaching and JoCoR retain two independently initialized two-layer MLPs during training; evaluation uses one network without ensembling their predictions.

The second group comprises structure-assisted methods evaluated under the graph-free inference protocol of this study: PosteL, ALS, SALS [60–62], Graph-MLP [4], and GSDN [14]. These methods may use graph information during

training but use only node attributes at inference. PosteL integrates its official posterior-target construction path with iterative pseudo-labeling disabled; the remaining methods retain their published core objectives or mechanisms under our data splits, noise realizations, and common MLP student. Appendix G reports the implementation provenance, protocol alignment, and method-specific parameters.

The graph-teacher extension compares graph-to-MLP knowledge-transfer routes under a common teacher–student protocol. The basic response-distillation baseline follows the GNN-to-MLP framework of GLNN [3]. We additionally compare FF-G2M, KRD, VQGraph, and the granular-ball consensus teacher [6, 7, 26]. FF-G2M is reimplemented with common teacher and student backbones while retaining its low- and high-frequency distillation objectives; KRD and VQGraph retain their reliable-knowledge selection and soft-codebook mechanisms. Both teacher and student are retrained for every seed, and the final student uses only node attributes.

4.4. Implementation details

All datasets were loaded through PyTorch Geometric using the NoisyGL data interface. GRACE pretraining used row-normalized node attributes. Downstream models used the node attributes supplied by NoisyGL. Graph-convolutional models additionally used its adjacency matrix, with normalization and self-loops handled within the graph-convolution layers.

All basic methods used the same two-layer MLP backbone with a hidden width of 64 and a dropout rate of 0.5. Dual-network methods used two independently initialized copies of this backbone. Models in this comparison group were trained for 200 epochs with Adam, a learning rate of 0.01, and weight decay of 5×10^{-4}. The checkpoint with the highest validation accuracy was retained, and no patience-based early stopping was used. The same validation-accuracy criterion was used for teacher checkpoint selection. Small-loss, Co-teaching, and JoCoR set their forget-rate schedules from the known synthetic noise rate; the other basic methods did not use this information.

GB-LCSS used $B_{\max} = 64$, $n_{\min} = 32$, $\delta = 0.05$, $s_{\min} = 2$, and $\beta = 0.5$ in all experiments. The 2-means procedure used Euclidean distance and k-means++ initialization. The downstream seed also controlled the 2-means initialization, and each candidate split used 10 initializations. GRACE used label-free, dataset-level configurations, and its encoder was frozen after pretraining. The distillation temperature was $\tau = 1$. Appendix G gives the complete parameters and their sources.

4.5. Statistical analysis

We report the mean test accuracy and sample standard deviation over five paired seeds. The main experiments used one pretrained representation H for each dataset. The five paired runs varied the data split, noise realization, downstream initialization, and, for GB-based methods, the 2-means initialization. Appendix D.3 provides a diagnostic across GRACE pretraining seeds. For each dataset and noise setting, we used a two-sided paired t-test, implemented as a one-sample t-test on the five same-seed accuracy differences, and computed 95% confidence intervals from a t distribution with four degrees of freedom. Holm correction was applied separately to three families of 25 comparisons: GB-LCSS versus random size-matched grouping, MLP+CE, and JoCoR. These families represent the grouping mechanism control, the standard MLP baseline, and a strong noisy-label baseline, respectively. These tests quantify the stability of paired differences across the five downstream seeds. Appendix B reports the complete paired results.

5. Results and analysis

5.1. Comparison with noise-robust methods using the same deployed MLP architecture

Table 2 compares GB-LCSS with seven noisy-label baselines under a common two-layer deployed MLP. All methods use the same labeled training nodes and receive only node attributes at inference. Values are test accuracy (%; mean ± sample standard deviation over five paired seeds), with the highest mean in each column shown in bold. Section 4.3 and Appendix G describe the comparison groups and implementation provenance.

Table 2. Test accuracy under uniform 0.4 label noise for methods sharing the same deployed MLP and graph-free inference protocol.

Method	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
MLP+CE	44.24±2.62	43.50±3.02	31.98±0.98	88.42±0.62	73.90±1.80
Small-loss	51.95±1.97	49.70±3.07	33.04±0.50	89.06±0.88	77.25±1.62
Co-teaching	46.83±1.78	46.24±2.89	34.39±0.24	88.80±0.97	78.21±1.00
JoCoR	51.72±1.30	49.91±3.37	35.70±0.32	89.38±0.61	79.57±0.53
SCE	43.60±1.86	42.09±3.47	29.29±0.45	88.37±0.53	76.08±3.11
ELR	44.28±1.38	43.17±3.95	32.14±0.67	88.29±0.73	73.93±2.12
LogitClip	47.50±1.50	46.57±3.52	34.69±1.17	88.40±0.21	74.98±2.38
GB-LCSS	**57.49±0.76**	**55.24±2.04**	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

Under uniform 0.4 noise, GB-LCSS attains the highest mean accuracy on all five datasets, exceeding the strongest baseline on each dataset by 1.33–5.54 percentage points.

Across the full noise spectrum, GB-LCSS achieves the highest or tied-highest mean among the seven baselines in 24 of 25 settings. Its mean gains over MLP+CE and JoCoR are 7.74 and 3.46 percentage points, respectively. The only exception is Coauthor-CS under uniform 0.2 noise, where GB-LCSS trails the strongest baseline by 0.27 points (Fig. 2; Appendices A and B provide the full results and paired tests).

Figure 3 complements these relative gains with absolute accuracy trajectories under increasing uniform noise. GB-LCSS leads the three comparison methods in 14 of the 15 dataset–noise combinations. As ρ increases from 0.2 to 0.6, its accuracy declines less than those of MLP+CE and JoCoR on four of the five datasets, with CiteSeer as the exception. Thus, the advantage is not confined to one noise rate and is accompanied by slower degradation on most datasets.

On the six supplementary datasets, GB-LCSS has a higher five-seed mean than the strongest same-architecture baseline on three datasets and than random size-matched grouping on all six. Appendix H gives the complete results.

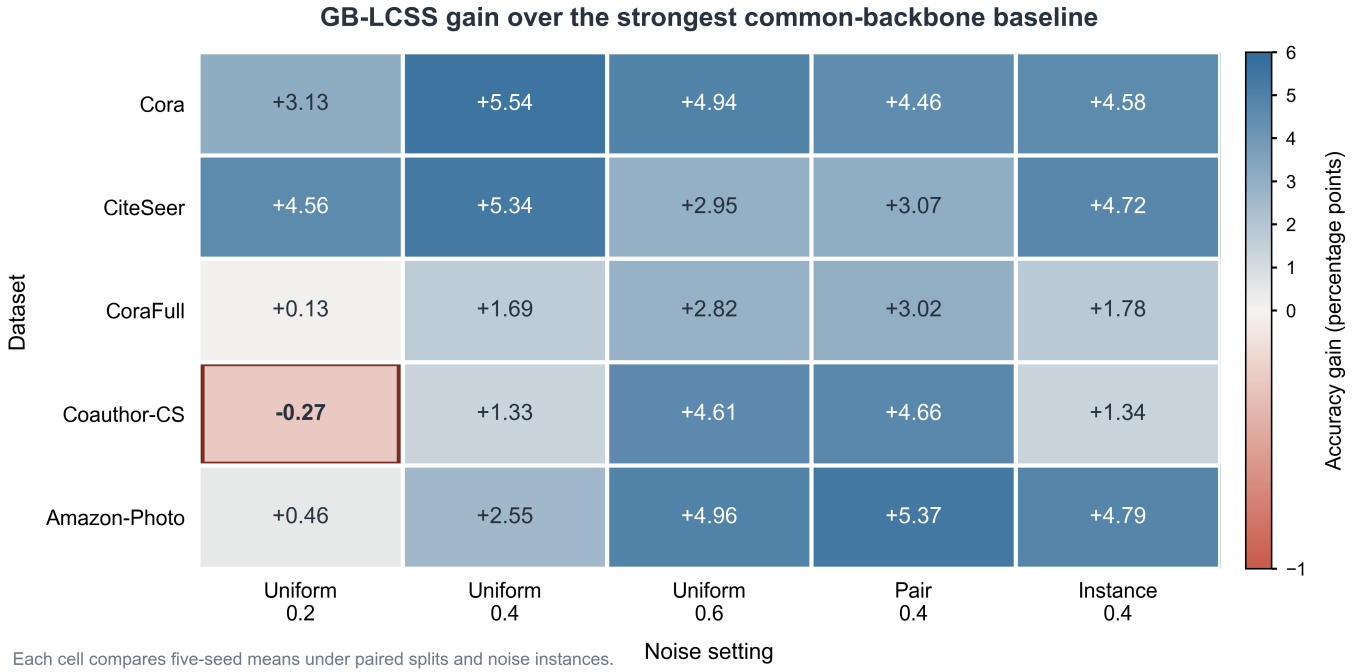


Figure 2. Accuracy gain of GB-LCSS over the strongest noise-robust baseline with the same deployment architecture. Each cell is the five-seed mean accuracy of GB-LCSS minus the highest mean among the seven baselines under the same dataset and noise setting, in percentage points. Blue and red denote positive and negative gains, respectively.

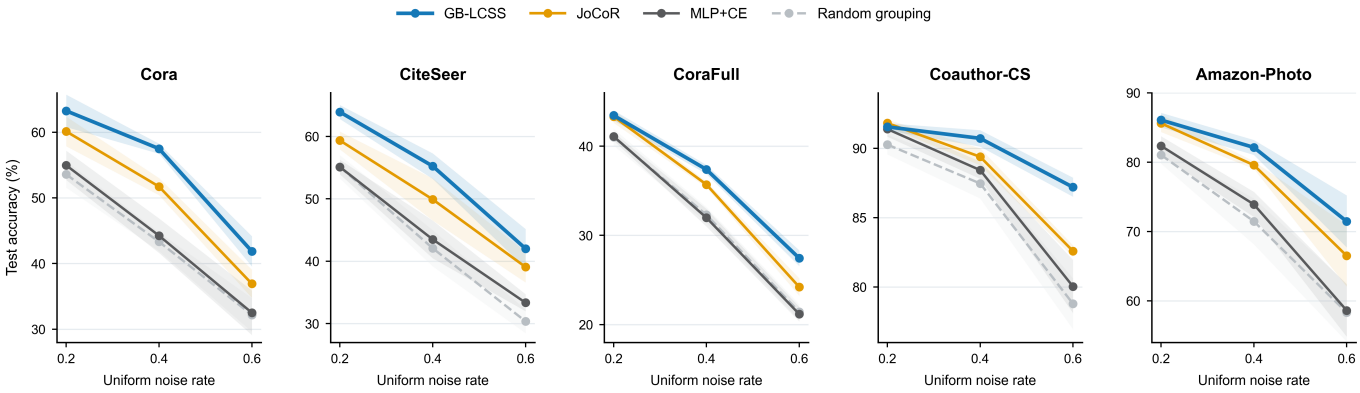


Figure 3. Test accuracy under uniform label noise. Solid lines show the means over five paired seeds, and shaded regions show sample standard deviations; the vertical scale is set independently in each panel. Random size-matched grouping randomizes node membership while preserving the number and size distribution of granular balls.

5.2. Structure-assisted methods for graph-free inference

Table 3 compares alternative ways of using graph structure during training while retaining node-attribute-only inference. Values are test accuracy (%; mean ± sample standard deviation over five seeds), with the highest mean in each column shown in bold. Section 4.3 and Appendix G describe the comparison protocol and implementation provenance.

Table 3. Test accuracy of structure-assisted methods with graph-free inference under uniform 0.4 label noise.

Method	Graph information used in training	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
PosteL	Structure-assisted soft labels	47.09±1.81	45.84±2.48	36.51±1.01	84.74±1.51	80.59±0.94
ALS	Neighborhood label smoothing	45.20±1.24	43.77±2.76	31.91±1.05	88.55±0.86	73.41±2.75
SALS	Structure-aware label smoothing	48.27±2.50	47.20±1.69	37.06±0.78	88.85±0.82	77.86±0.79
Graph-MLP	Multi-hop structural supervision	**63.23±3.07**	**57.23±1.85**	37.05±0.91	88.95±0.43	77.05±1.24
GSDN	Neighborhood self-distillation	55.97±4.29	50.36±3.39	35.80±0.66	86.55±0.78	73.31±2.00
GB-LCSS	Frozen-representation granular-ball consensus	57.49±0.76	55.24±2.04	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

Across the 25 settings evaluated under the common protocol, GB-LCSS exceeds PosteL, ALS, and SALS by 5.69, 7.71, and 4.69 percentage points in mean accuracy, respectively. The corresponding differences are positive in 24 of 25, all 25, and 24 of 25 settings. Appendix F reports the complete results.

The comparison with Graph-MLP is dataset dependent. Graph-MLP leads by 5.74 and 1.99 percentage points on Cora and CiteSeer, whereas GB-LCSS leads by 0.34, 1.76, and 5.07 points on CoraFull, Coauthor-CS, and Amazon-Photo.

5.3. Ablation and mechanism analysis

5.3.1. Role of the local-consensus target

The random size-matched and self-vote controls isolate the contributions of local membership and leave-one-out target construction. Replacing the true partition with random membership lowers accuracy in all 25 settings, by 8.69 percentage points on average. Removing the current node's vote adds a smaller but consistently positive refinement of 0.08–1.03 points when averaged over the five noise settings of each dataset. Under uniform 0.4, GB-LCSS also outperforms uniform smoothing and training-prior smoothing on every dataset; Appendix D provides the complete results.

Figure 4 evaluates the probability error of the soft targets using the multiclass Brier score, with clean training labels as a post-hoc reference. For a target matrix Y^{tar}, we use

$$\mathrm{BS}(Y^{\mathrm{tar}}) = \frac{1}{|\mathcal{V}_L|} \sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} \left(Y^{\mathrm{tar}}_{vc} - \mathbf{1}[y_v = c] \right)^2,$$

where y_v is the clean label of training node v. Across all 25 dataset–noise settings, the five-seed mean lies below the equality line. Relative to the original noisy one-hot targets, the GB-LCSS targets reduce the mean Brier score by 0.06–0.51, with the largest improvements on Coauthor-CS and Amazon-Photo.

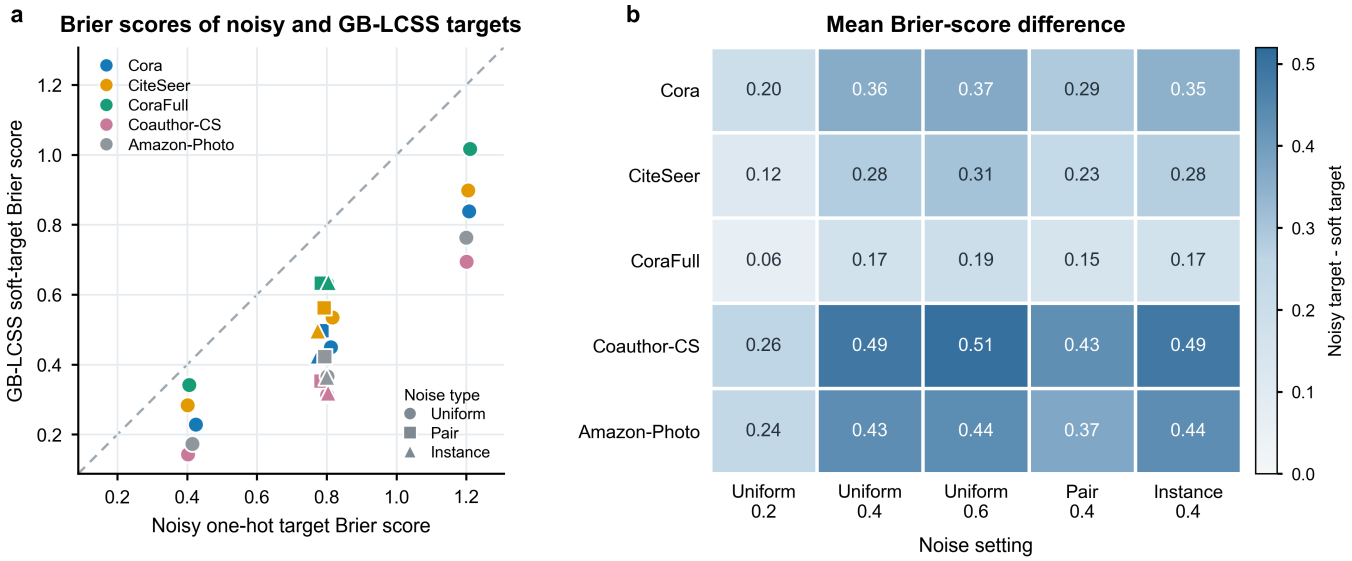


Figure 4. Post-hoc quality diagnostic for the GB-LCSS soft targets. (a) The horizontal and vertical axes show the multiclass Brier scores of the original noisy one-hot targets and the GB-LCSS targets, respectively. Each marker is the five-seed mean for one dataset–noise setting; colors identify datasets, and circles, squares, and triangles denote uniform, pair, and instance-dependent noise. The dashed line denotes equality. (b) Each cell is the original-target score minus the soft-target score; positive values indicate lower squared probabilistic error for the soft targets.

5.3.2. Role of the granular-ball partition

The kNN and k-means alternatives use the same leave-one-out target as the main method. For each seed, kNN matches its neighborhood size to the mean number of training votes per ball and excludes the query node. The number of k-means clusters matches the number of granular balls for that seed, and the current node is likewise removed from its cluster target. Relative to the better of the two alternatives, granular balls differ by +0.27, −0.94, −3.11, −0.29, and −0.11 percentage points on the five full-spectrum datasets. Except on CoraFull, the gap among the three grouping implementations remains below one percentage point. Local-consensus supervision therefore does not depend on a unique grouping rule.

We retain granular balls as the default regional interface in GB-LCSS because the budgeted recursive partition automatically determines a bounded set of disjoint regions and supports shared regional statistics. In the matched k-means control, by contrast, the cluster count is supplied by the number of granular balls produced for the same seed. kNN maintains overlapping and potentially asymmetric neighborhoods for individual training nodes and aggregates their labels separately.

5.4. Clean-label control

We also test whether GB-LCSS improves classification with clean training labels by setting ρ=0 and keeping the data splits and other training settings unchanged. The accuracy differences between GB-LCSS and MLP+CE on Cora, CiteSeer, CoraFull, Coauthor-CS, and Amazon-Photo are −0.24, +2.00, −0.38, −0.87, and −1.75 percentage points, respectively. Except on CiteSeer, GB-LCSS does not improve accuracy with clean labels.

This pattern indicates that the observed advantage of GB-LCSS over MLP+CE is concentrated in the noisy-label settings evaluated here.

5.5. Training-time representation and grouping costs

All timing and GPU-memory measurements were collected on the same workstation with an AMD Radeon RX 7700 XT, using Python 3.12.10 and PyTorch 2.9.1+rocm7.2.1. GB-LCSS and MLP+CE use the same student and therefore

have the same parameter count, inference input, and forward complexity. The additional cost comes from GRACE representation learning and granular-ball construction before student training. To illustrate the one-time representation-learning cost, we report the recorded GRACE pretraining time and peak GPU memory for Cora, CiteSeer, and CoraFull. Pretraining times are averaged over three pretraining seeds, and memory values are the largest recorded peaks across the three runs. Pretraining takes 0.07, 0.13, and 74.48 minutes, and peak GPU memory usage is 0.40, 0.67, and 12.73 GiB, respectively. The frozen representation can be reused for every noise setting and downstream seed on the same dataset.

Figure 5 quantifies the engineering trade-off among granular balls, exact kNN, and k-means. In the seed-0 implementation profile, granular balls have lower end-to-end construction time on three of the five datasets and require less grouping storage than exact kNN on CoraFull, Coauthor-CS, and Amazon-Photo. Together with the accuracy results in Section 5.3.2, these measurements motivate the granular-ball interface through its disjoint, reusable, fixed-budget regions; the alternative interfaces remain competitive and can be cheaper on individual datasets.

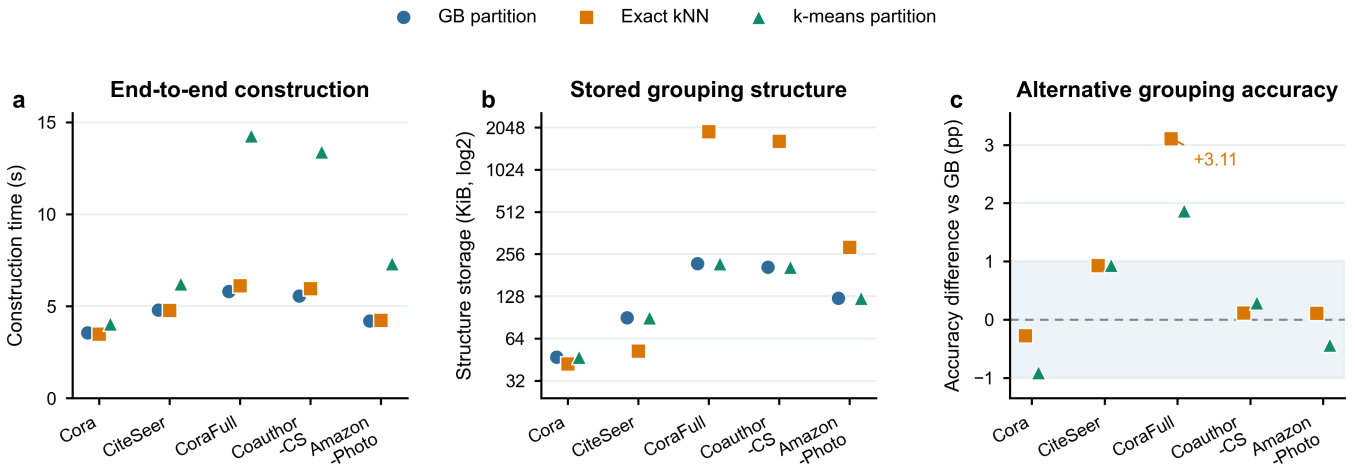


Figure 5. Accuracy and cost of granular-ball, exact-kNN, and k-means grouping. (a) End-to-end construction time. (b) Stored grouping structure. (c) Test-accuracy differences of exact kNN and k-means relative to granular balls; positive values favor the alternative, and the blue region denotes ±1 percentage point. Time and storage are seed-0 measurements; accuracy is the five-seed mean under uniform 0.4 noise.

5.6. Discussion

Across the full-spectrum and supplementary evaluations, geometry-derived grouping is more effective than random size-matched grouping. Gains over strong noisy-label classifiers and alternative grouping interfaces vary across datasets, and clean-label training shows no consistent additional benefit. These results support local-consensus soft-target construction for noisy-label learning, while showing that the choice of grouping interface remains dataset dependent.

6. Graph-teacher distillation extension

6.1. From training-node targets to distillation over all nodes

Basic GB-LCSS directly supervises only the training nodes. The distillation extension broadens this coverage by training a GCN teacher on the same targets and using its predictions over all nodes to supervise the MLP student. The teacher is used only during training, so the deployed model remains the same attribute-only MLP.

We evaluated the extension on Cora, CiteSeer, CoraFull, Coauthor-CS, and Amazon-Photo under uniform 0.4 and 0.6 to compare different forms of teacher supervision and knowledge transfer.

6.2. Comparison of graph-knowledge transfer routes

To compare different knowledge-transfer mechanisms, Table 4 evaluates the routes under the common teacher–student protocol defined in Section 4.3. FF-G2M transfers low-frequency neighborhood knowledge and high-frequency edge-difference knowledge; KRD selects reliable knowledge according to the teacher's stability under feature perturbations; and VQGraph represents local structure through a soft codebook [6, 7, 26]. The granular-ball consensus route trains a GCN(X) teacher with the GB-LCSS target in Eq. (14). All routes use the same data splits and noise realizations, and each route independently trains its teacher and an identically configured two-layer MLP student for every seed.

Values are test accuracy (%; mean ± standard deviation over five seeds), with the highest mean in each column shown in bold. All distillation routes use the same two-layer student, which reads only node attributes at inference. Appendix G reports the implementation provenance and protocol alignment of each route.

Table 4. Test accuracy of basic GB-LCSS and graph-to-MLP knowledge-transfer routes under a common teacher–student protocol and uniform label-noise rates of 0.4 and 0.6.

(a) Uniform 0.4

Training route	**Cora**	**CiteSeer**	**CoraFull**	**Coauthor-CS**	**Amazon-Photo**
Basic GB-LCSS (without distillation)	57.49±0.76	55.24±2.04	37.39±0.51	90.71±0.59	82.12±1.08
GCN(X)+CE teacher	77.95±2.62	66.09±1.38	**58.74±0.82**	93.20±0.09	89.86±0.77
FF-G2M	77.65±2.97	65.80±0.49	56.58±0.93	91.58±0.24	84.82±0.84
KRD	71.77±2.20	64.96±1.03	47.12±0.95	91.39±0.49	85.17±0.94
VQGraph	78.89±2.34	65.64±1.72	58.52±0.77	93.25±0.09	**90.54±0.85**
Granular-ball consensus teacher	**81.35±1.05**	**69.27±1.57**	57.29±0.24	**93.83±0.21**	90.40±0.77

(b) Uniform 0.6

Training route	**Cora**	**CiteSeer**	**CoraFull**	**Coauthor-CS**	**Amazon-Photo**
Basic GB-LCSS (without distillation)	41.86±2.30	42.02±3.14	27.43±0.89	87.19±0.69	71.45±3.75
GCN(X)+CE teacher	65.50±3.28	53.30±4.42	51.63±2.02	91.13±0.58	84.80±1.48
FF-G2M	63.63±2.16	52.32±4.20	45.78±2.90	86.50±1.01	73.08±2.82
KRD	61.28±1.17	54.17±3.14	41.61±2.43	89.80±0.29	80.30±0.51
VQGraph	65.75±2.72	53.34±4.47	52.01±0.65	90.71±1.24	84.77±2.19
Granular-ball consensus teacher	**71.34±3.91**	**58.93±4.62**	**52.32±1.26**	**92.40±0.65**	**86.81±1.94**

Training the GCN with GB-LCSS targets yields higher mean student accuracy than training it with cross-entropy in nine of the ten settings. The distilled student also improves on basic GB-LCSS in all ten settings, by 3.12–23.86 percentage points under uniform 0.4 and 5.21–29.48 points under uniform 0.6, and achieves the highest mean among the six routes in Table 4 in eight settings. The two settings in which the consensus-teacher student does not achieve the highest mean both occur under uniform 0.4: it trails GCN(X)+CE by 1.45 points on CoraFull and VQGraph by 0.14 points on Amazon-Photo. The teacher is discarded after training, so every deployed student still reads only node attributes.

7. Conclusion

This study addresses transductive node classification with noisy training labels when graph structure is available during training but inference is limited to node attributes. GB-LCSS forms disjoint, label-agnostic regions in a graph contrastive representation and converts leave-one-out local consensus into node-specific targets for the deployed MLP. Across five datasets and five noise settings, it achieves the highest or tied-highest five-seed mean in 24 of 25 settings among seven noise-robust baselines using the same deployed MLP architecture. Averaged over these settings, GB-LCSS also has higher mean accuracy than PosteL, ALS, and SALS under the common protocol.

Granular balls organize supervision into a bounded set of disjoint regions. Within each ball, a single label histogram is reused by subtracting each training node's own vote. Alternative neighborhood and clustering rules also support local consensus. As a secondary extension, a graph teacher trained on the same targets supplies predictions over all nodes and improves the MLP student's mean accuracy under the evaluated protocol without changing the deployment interface.

The experiments use synthetic label noise, clean validation labels, and transductive evaluation on fixed graphs; target construction also depends on a pretrained and frozen graph representation. Future work should examine real annotation noise, model selection without clean validation labels, inductive inference for new nodes, and robustness across representation-pretraining runs.

8. Data and code availability

All graph datasets used in this study are public benchmarks. The anonymous supplementary materials submitted with the manuscript provide runnable code for GB-LCSS, the same-architecture baselines, random size-matched grouping, and the granular-ball consensus teacher, together with the configurations, frozen representations, seed-wise results, table snapshots, verification scripts, final figure files, and GRACE settings used in the reported experiments. For protocol-adapted comparison methods, the supplementary materials provide the configurations, implementation notes, and seed-wise results used in this study.

References

[1] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. ICLR, 2017.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. ICLR, 2018.

[3] S. Zhang, Y. Liu, Y. Sun, N. Shah, Graph-less neural networks: Teaching old MLPs new tricks via distillation, in: Proc. ICLR, 2022.

[4] Y. Hu, H. You, Z. Wang, Z. Wang, E. Zhou, Y. Gao, Graph-MLP: Node classification without message passing in graph, arXiv preprint arXiv:2106.04051, 2021.

[5] Y. Tian, C. Zhang, Z. Guo, X. Zhang, N. V. Chawla, Learning MLPs on graphs: A unified view of effectiveness, robustness, and efficiency, in: Proc. ICLR, 2023.

[6] L. Yang, Y. Tian, M. Xu, Z. Liu, S. Hong, W. Qu, W. Zhang, B. Cui, M. Zhang, J. Leskovec, VQGraph: Rethinking graph representation space for bridging GNNs and MLPs, in: Proc. ICLR, 2024.

[7] L. Wu, H. Lin, Y. Huang, S. Z. Li, Quantifying the knowledge in GNNs for reliable distillation into MLPs, in: Proc. ICML, 2023.

[8] C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning (still) requires rethinking generalization, Communications of the ACM 64 (3) (2021) 107–115.

[9] E. Dai, C. Aggarwal, S. Wang, NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs, in: Proc. ACM SIGKDD, 2021.

[10] S. Qian, H. Ying, R. Hu, J. Zhou, J. Chen, D. Z. Chen, J. Wu, Robust training of graph neural networks via noise governance, in: Proc. WSDM, 2023, pp. 607–615. https://doi.org/10.1145/3539597.3570369.

[11] K. Ding, X. Ma, Y. Liu, S. Pan, Divide and denoise: Empowering simple models for robust semi-supervised node classification against label noise, in: Proc. ACM SIGKDD, 2024, pp. 574–584.

[12] Y. Zhu, L. Feng, Z. Deng, Y. Chen, R. Amor, M. Witbrock, Robust node classification on graph data with graph and label noise, in: Proc. AAAI, 2024.

[13] Z. Wang, D. Sun, S. Zhou, H. Wang, J. Fan, L. Huang, J. Bu, NoisyGL: A comprehensive benchmark for graph neural networks under label noise, Advances in Neural Information Processing Systems 37 (2024) 38142–38170.

[14] L. Wu, J. Xia, H. Lin, Z. Gao, Z. Liu, G. Zhao, S. Z. Li, Teaching yourself: Graph self-distillation on neighborhood for node classification, arXiv preprint arXiv:2210.02097, 2022.

[15] Z. Wang, Z. Zhang, C. Zhang, Y. Ye, Training MLPs on graphs without supervision, in: Proc. WSDM, 2025.

[16] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, in: ICML Workshop on Graph Representation Learning and Beyond, 2020.

[17] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. W. Tsang, M. Sugiyama, Co-teaching: Robust training of deep neural networks with extremely noisy labels, in: Proc. NeurIPS, 2018.

[18] H. Wei, L. Feng, X. Chen, B. An, Combating noisy labels by agreement: A joint training method with co-regularization, in: Proc. CVPR, 2020.

[19] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, J. Bailey, Symmetric cross entropy for robust learning with noisy labels, in: Proc. ICCV, 2019.

[20] S. Liu, J. Niles-Weed, N. Razavian, C. Fernandez-Granda, Early-learning regularization prevents memorization of noisy labels, in: Proc. NeurIPS, 2020.

[21] X. Ye, X. Li, S. Dai, T. Liu, Y. Sun, W. Tong, Active negative loss functions for learning with noisy labels, in: Proc. NeurIPS, 2023.

[22] H. Wei, H. Zhuang, R. Xie, L. Feng, G. Niu, B. An, Y. Li, Mitigating memorization of noisy labels by clipping the model prediction, in: Proc. ICML, 2023.

[23] J. Li, R. Socher, S. C. H. Hoi, DivideMix: Learning with noisy labels as semi-supervised learning, in: Proc. ICLR, 2020.

[24] T. Kim, J. Ko, S. Cho, J. Choi, S.-Y. Yun, FINE samples for learning with noisy labels, in: Proc. NeurIPS, 2021.

[25] W. Pan, W. Wei, F. Zhu, Y. Deng, Enhanced sample selection with confidence tracking: Identifying correctly labeled yet hard-to-learn samples in noisy data, in: Proc. AAAI, 39 (19) (2025) 19795–19803. https://doi.org/10.1609/aaai.v39i19.34180.

[26] L. Wu, H. Lin, Y. Huang, T. Fan, S. Z. Li, Extracting low-/high-frequency knowledge from graph neural networks and injecting it into MLPs: An effective GNN-to-MLP distillation framework, in: Proc. AAAI, 2023.

[27] J. Chen, M. Bai, S. Chen, J. Gao, J. Zhang, J. Pu, SA-MLP: Distilling graph knowledge from GNNs into structure-aware MLP, Transactions on Machine Learning Research (2024).

[28] T. Wu, Z. Zhao, J. Wang, X. Bai, L. Wang, N. Wong, Y. Yang, Edge-free but structure-aware: Prototype-guided knowledge distillation from GNNs to MLPs, in: Proc. COLING, 2025, pp. 5656–5667.

[29] W. Lu, Z. Guan, W. Zhao, Y. Yang, AdaGMLP: AdaBoosting GNN-to-MLP knowledge distillation, in: Proc. ACM SIGKDD, 2024, pp. 2060–2071. https://doi.org/10.1145/3637528.3671699.

[30] W. Lu, Z. Guan, W. Zhao, Y. Yang, Y. Sun, Z. Liang, Y. Zhan, D. Tao, ProGMLP: A progressive framework for GNN-to-MLP knowledge distillation with efficient trade-offs, in: Proc. AAAI, 40 (29) (2026) 24088–24096. https://doi.org/10.1609/aaai.v40i29.39587.

[31] H. Pal, S. Malik, R. Patel, A. Malhotra, SALE-MLP: Structure aware latent embeddings for GNN to graph-free MLP distillation, in: Proc. IJCAI, 2025, pp. 6003–6011. https://doi.org/10.24963/ijcai.2025/668.

[32] Z. Yu, M. Lin, W. Li, Z. Yin, S. Xiao, S. Lu, Demystifying GNN-to-MLP knowledge transfer: Theoretical grounding and dual-stream distillation method, in: Proc. AAAI, 40 (33) (2026) 28005–28013. https://doi.org/10.1609/aaai.v40i33.40025.

[33] M. Kohn, M. Hoffmann, A. Scherp, Edge-splitting MLP: Node classification on homophilic and heterophilic graphs without message passing, in: Proc. Third Learning on Graphs Conference, PMLR 269 (2025) 11:1–11:21. https://proceedings.mlr.press/v269/kohn25a.html.

[34] S. Xia, Y. Liu, X. Ding, G. Wang, H. Yu, Y. Luo, Granular ball computing classifiers for efficient, scalable and robust learning, Information Sciences 483 (2019) 136–152.

[35] F. Dunkin, X. Li, H. Li, G. Wu, C. Hu, S. S. Ge, MgCNL: A sample separation approach via multi-granularity balls for fault diagnosis with the interference of noisy labels, IEEE Transactions on Automation Science and Engineering 22 (2025) 7748–7761. https://doi.org/10.1109/TASE.2024.3469000.

[36] F. Dunkin, X. Li, C. Hu, G. Wu, H. Li, X. Lu, Z. Zhang, Like draws to like: A multi-granularity ball-intra fusion approach for fault diagnosis models to resists misleading by noisy labels, Advanced Engineering Informatics 60 (2024) 102425. https://doi.org/10.1016/j.aei.2024.102425.

[37] S. Xia, S. Zheng, G. Wang, X. Gao, B. Wang, Granular ball sampling for noisy label classification or imbalanced classification, IEEE Transactions on Neural Networks and Learning Systems 34 (4) (2023) 2144–2155. https://doi.org/10.1109/TNNLS.2021.3105984.

[38] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, T. Eliassi-Rad, Collective classification in network data, AI Magazine 29 (3) (2008) 93–106.

[39] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, in: Relational Representation Learning Workshop, NeurIPS, 2018. https://www.cs.cit.tum.de/daml/gnn-benchmark/.

[40] Z. Wang, Y. Bei, S. Zhou, Z. Zhou, J. Fan, H. Xue, H. Wang, J. Bu, Learning from graph: Mitigating label noise on graph through topological feature reconstruction, in: Proc. CIKM, 2025, pp. 3261–3270. https://doi.org/10.1145/3746252.3761185.

[41] K. Li, J. Sun, J. Lou, Z. Feng, H. Zhou, C. Wu, G. Xue, W. Zhao, J. Li, Leveraging peer-informed label consistency for robust graph neural networks with noisy labels, in: Proc. IJCAI, 2025, pp. 5598–5606. https://doi.org/10.24963/ijcai.2025/623.

[42] K. Li, J. Lou, J. Sun, H. Zeng, W. Li, C. Wu, Y. Luo, W. Zhao, S. Du, J. Li, GD^2: Robust graph learning under label noise via dual-view prediction discrepancy, Advances in Neural Information Processing Systems 38 (2025).

[43] Z. Zhou, Z. Ding, J. Shi, L. Qing, S. Shen, TINED: GNNs-to-MLPs by teacher injection and Dirichlet energy distillation, in: Proc. ICML, PMLR 267 (2025) 78616–78632. https://proceedings.mlr.press/v267/zhou25a.html.

[44] S. Li, B. Zhang, J. Song, G. Ruan, C. Wang, J. Xie, Graph neural networks with coarse- and fine-grained division for mitigating label noise and sparsity, Neural Networks 187 (2025) 107338. https://doi.org/10.1016/j.neunet.2025.107338.

[45] B. Hu, G. Wen, Y. Huang, Y. Liu, X. Zhu, Structure-aware multi-teacher distillation for noise node classification, Information Processing & Management 63 (7) (2026) 104877. https://www.sciencedirect.com/science/article/pii/S0306457326002682.

[46] Y. Li, J. Yin, L. Chen, Unified robust training for graph neural networks against label noise, in: Proc. PAKDD, Lecture Notes in Computer Science 12712 (2021) 528–540. https://doi.org/10.1007/978-3-030-75762-5_42.

[47] Y. Wu, J. Yao, X. Xia, J. Yu, R. Wang, B. Han, T. Liu, Mitigating label noise on graphs via topological sample selection, in: Proc. ICML, PMLR 235 (2024) 53944–53972. https://proceedings.mlr.press/v235/wu24ae.html.

[48] J. Yang, Q. Liu, X. Cao, F. Li, X. Yang, G-GBC: A Gaussian mixture-based granular ball computing method for robust CNN classification under noisy labels, Knowledge-Based Systems 341 (2026) 115754. https://doi.org/10.1016/j.knosys.2026.115754.

[49] Y. Li, X. Ouyang, J. Zhang, C. Pan, L. Ren, X. Yang, Beneficial noise learning for open intent classification via granular-ball representation, Pattern Recognition 177 (2026) 113283. https://doi.org/10.1016/j.patcog.2026.113283.

[50] Z. Liu, H. Yu, X. Luo, Mitigating label noise in graph learning with information bottleneck-guided aggregation and adversarial consistency, Pattern Recognition 180 (2026) 114271. https://doi.org/10.1016/j.patcog.2026.114271.

[51] S. Li, B. Zhang, M. Cao, J. Song, X. Cheng, C. Wang, HALF: A homophily-aware loss fusion for robust learning under label noise in heterophilic graphs, Information Sciences 754 (2026) 123714. https://doi.org/10.1016/j.ins.2026.123714.

[52] X. Peng, P. Wang, S. Xia, C. Wang, W. Chen, VPGB: A granular-ball based model for attribute reduction and classification with label noise, Information Sciences 611 (2022) 504–521. https://doi.org/10.1016/j.ins.2022.08.066.

[53] J. Zhang, X. Sun, B. Huang, T. Wang, H. Li, X. Jia, GEAF: A label noise filtering method based on adaptive partitioning of granular ellipsoids, Information Sciences 742 (2026) 123324. https://doi.org/10.1016/j.ins.2026.123324.

[54] S. Xia, X. Ma, Z. Liu, C. Liu, S. Zhao, G. Wang, Graph coarsening via supervised granular-ball for scalable graph neural network training, in: Proc. AAAI, 39 (12) (2025) 12872–12880. https://doi.org/10.1609/aaai.v39i12.33404.

[55] S. Xia, G. Wang, G. Xu, S. Zhao, G. Wang, GBGC: Efficient and adaptive graph coarsening via granular-ball computing, in: Proc. IJCAI, 2025, pp. 3489–3497. https://doi.org/10.24963/ijcai.2025/388.

[56] J. Ni, S. Xia, Y. Guan, G. Xu, L. Chen, Y. Liu, Y. Wang, Multi-granularity graph refinement via granular ball for graph classification, Neurocomputing 686 (2026) 133713. https://doi.org/10.1016/j.neucom.2026.133713.

[57] Z. Wang, T. Zhang, S. Xia, L. Lin, G. Wang, GBRAIN: Combating textual label noise by granular-ball based robust training, in: Proc. ACM ICMR, 2024, pp. 357–365. https://doi.org/10.1145/3652583.3658084.

[58] Y. Liu, Z. Wu, Z. Lu, C. Nie, G. Wen, Y. Zhu, X. Zhu, Noisy node classification by bi-level optimization based multi-teacher distillation, in: Proc. AAAI, 39 (18) (2025) 19033–19040. https://doi.org/10.1609/aaai.v39i18.34095.

[59] M. Zhou, X. Han, W. Wei, G. Yan, Graph topology adaptive judgment against node label noise, Knowledge-Based Systems 328 (2025) 114162. https://doi.org/10.1016/j.knosys.2025.114162.

[60] J. Heo, M. Park, D. Kim, Posterior label smoothing for node classification, in: Proc. AAAI, 40 (26) (2026) 21708–21716. https://doi.org/10.1609/aaai.v40i26.39321.

[61] K. Zhou, S.-H. Choi, Z. Liu, N. Liu, F. Yang, R. Chen, L. Li, X. Hu, Adaptive label smoothing to regularize large-scale graph training, in: Proc. 2023 SIAM International Conference on Data Mining, 2023, pp. 55–63. https://doi.org/10.1137/1.9781611977653.ch7.

[62] Y. Wang, Y. Cai, Y. Liang, W. Wang, H. Ding, M. Chen, J. Tang, B. Hooi, Structure-aware label smoothing for graph neural networks, arXiv preprint arXiv:2112.00499 (2021). https://arxiv.org/abs/2112.00499.

[63] Q. Li, X. Li, D. Li, J. Wang, Prototype-guided supervision for graph learning with noisy and sparse labels, in: Proc. AAAI, 40 (27) (2026) 23105–23113. https://doi.org/10.1609/aaai.v40i27.39477.

Supplementary material

Appendix A. Full noise spectrum for the principal comparisons

Appendix A.1. Cora

Table A.1. Full noise-spectrum results for the principal methods on Cora. Values are test accuracy (%; mean ± standard deviation over five seeds). Bold indicates the highest mean under each noise setting.

Method	Uniform 0.2	Uniform 0.4	Uniform 0.6	Pair 0.4	Instance 0.4
MLP+CE	54.96±2.22	44.24±2.62	32.51±3.38	42.83±2.19	45.33±2.22
Small-loss	59.84±1.29	51.95±1.97	36.72±1.09	46.99±2.77	51.82±0.94
Co-teaching	58.62±1.97	46.83±1.78	33.29±3.48	44.26±2.30	47.35±2.24
JoCoR	60.12±2.25	51.72±1.30	36.92±2.14	45.91±2.38	52.26±1.01
SCE	53.57±2.17	43.60±1.86	32.66±3.78	41.40±2.04	43.29±4.35
ELR	52.95±2.38	44.28±1.38	32.82±3.51	41.83±1.75	45.93±2.26
LogitClip	58.63±1.35	47.50±1.50	34.44±2.67	44.90±1.21	48.09±1.73
Random size-matched grouping	53.57±1.99	43.31±1.40	32.18±2.54	39.28±1.34	43.34±1.70
GB-LCSS	**63.25±2.46**	**57.49±0.76**	**41.86±2.30**	**51.45±2.80**	**56.84±1.67**

Appendix A.2. CiteSeer

Table A.2. Full noise-spectrum results for the principal methods on CiteSeer. Values are test accuracy (%; mean ± standard deviation over five seeds). Bold indicates the highest mean under each noise setting.

Method	Uniform 0.2	Uniform 0.4	Uniform 0.6	Pair 0.4	Instance 0.4
MLP+CE	55.10±1.17	43.50±3.02	33.35±1.36	45.15±1.67	44.33±2.04
Small-loss	59.26±1.94	49.70±3.07	37.73±2.86	48.16±2.56	50.75±3.33
Co-teaching	57.05±0.82	46.24±2.89	34.19±2.92	46.25±2.29	46.81±1.68
JoCoR	59.36±1.43	49.91±3.37	39.07±2.47	48.63±2.35	50.86±2.45
SCE	52.12±3.28	42.09±3.47	30.59±2.83	42.22±1.56	40.92±4.23
ELR	53.62±1.48	43.17±3.95	30.80±2.70	42.69±2.43	41.94±3.27
LogitClip	58.75±1.44	46.57±3.52	34.47±1.69	45.78±1.06	47.41±2.64
Random size-matched grouping	55.13±1.85	42.08±2.88	30.36±1.88	43.92±1.67	43.07±2.00
GB-LCSS	**63.92±1.12**	**55.24±2.04**	**42.02±3.14**	**51.70±2.14**	**55.58±2.54**

Appendix A.3. CoraFull

Table A.3. Full noise-spectrum results for the principal methods on CoraFull. Values are test accuracy (%; mean ± standard deviation over five seeds). Bold indicates the highest mean under each noise setting.

Method	Uniform 0.2	Uniform 0.4	Uniform 0.6	Pair 0.4	Instance 0.4
MLP+CE	41.11±0.46	31.98±0.98	21.16±0.81	28.14±1.17	31.33±0.85
Small-loss	41.58±0.94	33.04±0.50	21.51±0.98	30.15±1.11	32.90±0.87
Co-teaching	41.70±1.05	34.39±0.24	22.84±1.38	30.28±0.52	33.39±0.90
JoCoR	43.33±0.70	35.70±0.32	24.20±0.98	31.45±0.80	35.17±0.51
SCE	37.93±0.72	29.29±0.45	17.79±1.33	26.98±0.51	29.68±1.06
ELR	41.08±0.53	32.14±0.67	21.32±0.98	28.91±1.44	31.52±0.83
LogitClip	41.24±0.78	34.69±1.17	24.62±0.72	31.23±1.84	34.19±0.17
Random size-matched grouping	41.02±0.92	32.30±0.85	21.41±0.84	28.49±0.78	31.43±0.79
GB-LCSS	**43.46±0.57**	**37.39±0.51**	**27.43±0.89**	**34.46±1.23**	**36.95±0.53**

Appendix A.4. Coauthor-CS

Table A.4. Full noise-spectrum results for the principal methods on Coauthor-CS. Values are test accuracy (%; mean ± standard deviation over five seeds). Bold indicates the highest mean under each noise setting.

Method	Uniform 0.2	Uniform 0.4	Uniform 0.6	Pair 0.4	Instance 0.4
MLP+CE	91.38±0.54	88.42±0.62	80.03±1.91	75.91±1.34	87.44±1.15
Small-loss	91.50±0.49	89.06±0.88	81.65±0.91	80.13±1.50	88.97±1.02
Co-teaching	91.70±0.29	88.80±0.97	81.88±0.72	79.06±1.30	88.95±0.97
JoCoR	**91.81±0.25**	89.38±0.61	82.58±0.41	81.02±1.62	89.17±0.85
SCE	90.63±0.47	88.37±0.53	79.43±1.45	79.77±2.51	87.99±0.56
ELR	91.21±0.60	88.29±0.73	80.54±1.28	78.03±1.48	87.78±0.94
LogitClip	91.32±0.43	88.40±0.21	79.96±1.64	81.57±0.53	88.60±0.75
Random size-matched grouping	90.26±0.70	87.46±1.07	78.78±1.82	75.99±0.60	86.87±1.56
GB-LCSS	91.54±0.26	**90.71±0.59**	**87.19±0.69**	**86.24±1.18**	**90.51±0.43**

Appendix A.5. Amazon-Photo

Table A.5. Full noise-spectrum results for the principal methods on Amazon-Photo. Values are test accuracy (%; mean ± standard deviation over five seeds). Bold indicates the highest mean under each noise setting.

Method	Uniform 0.2	Uniform 0.4	Uniform 0.6	Pair 0.4	Instance 0.4
MLP+CE	82.34±1.39	73.90±1.80	58.60±3.77	62.61±4.31	69.13±3.00
Small-loss	84.96±0.58	77.25±1.62	61.40±3.72	66.70±5.73	73.00±1.25
Co-teaching	85.21±0.51	78.21±1.00	63.29±5.00	67.24±6.97	73.59±1.19
JoCoR	85.60±1.26	79.57±0.53	66.48±4.43	66.94±6.36	74.80±1.79
SCE	84.74±0.94	76.08±3.11	57.82±4.07	66.51±6.96	73.08±2.54
ELR	82.31±1.98	73.93±2.12	59.32±4.53	66.07±5.25	69.41±2.70
LogitClip	83.78±1.02	74.98±2.38	62.42±4.37	68.52±4.20	72.90±1.05
Random size-matched grouping	81.03±1.23	71.45±3.29	58.30±4.17	62.56±4.24	67.56±2.89
GB-LCSS	**86.07±1.05**	**82.12±1.08**	**71.45±3.75**	**73.89±3.73**	**79.59±0.88**

Appendix B. Paired statistical tests

Mean differences and 95% confidence intervals are expressed in percentage points. Each setting uses a two-sided paired t-test, implemented as a one-sample t-test on the five same-seed accuracy differences; confidence intervals are computed from a t distribution with four degrees of freedom. Holm correction is applied separately within three comparison families of 25 tests each.

Table B.1. Setting-wise paired statistics for GB-LCSS versus random size-matched grouping, MLP+CE, and JoCoR.

Dataset	Noise	Comparison	Mean difference	95% CI	Positive seeds	Holm-adjusted p
Cora	Uniform 0.2	GB-LCSS−random size-matched grouping	+9.68	[+7.16, +12.20]	5/5	0.006
Cora	Uniform 0.2	GB-LCSS−MLP+CE	+8.30	[+5.59, +11.00]	5/5	0.011
Cora	Uniform 0.2	GB-LCSS−JoCoR	+3.13	[+0.55, +5.72]	5/5	0.112
Cora	Uniform 0.4	GB-LCSS−random size-matched grouping	+14.17	[+12.71, +15.63]	5/5	¡0.001
Cora	Uniform 0.4	GB-LCSS−MLP+CE	+13.25	[+10.81, +15.68]	5/5	0.002
Cora	Uniform 0.4	GB-LCSS−JoCoR	+5.77	[+3.60, +7.95]	5/5	0.031
Cora	Uniform 0.6	GB-LCSS−random size-matched grouping	+9.68	[+6.39, +12.96]	5/5	0.010
Cora	Uniform 0.6	GB-LCSS−MLP+CE	+9.35	[+4.54, +14.16]	5/5	0.011
Cora	Uniform 0.6	GB-LCSS−JoCoR	+4.94	[+1.79, +8.09]	5/5	0.082
Cora	Pair 0.4	GB-LCSS−random size-matched grouping	+12.17	[+7.23, +17.11]	5/5	0.012
Cora	Pair 0.4	GB-LCSS−MLP+CE	+8.62	[+5.74, +11.50]	5/5	0.011
Cora	Pair 0.4	GB-LCSS−JoCoR	+5.54	[+2.90, +8.18]	5/5	0.060
Cora	Instance 0.4	GB-LCSS−random size-matched grouping	+13.51	[+12.08, +14.94]	5/5	¡0.001
Cora	Instance 0.4	GB-LCSS−MLP+CE	+11.51	[+9.50, +13.53]	5/5	0.002
Cora	Instance 0.4	GB-LCSS−JoCoR	+4.58	[+2.27, +6.89]	5/5	0.062
CiteSeer	Uniform 0.2	GB-LCSS−random size-matched grouping	+8.79	[+6.87, +10.71]	5/5	0.004
CiteSeer	Uniform 0.2	GB-LCSS−MLP+CE	+8.81	[+7.00, +10.63]	5/5	0.003
CiteSeer	Uniform 0.2	GB-LCSS−JoCoR	+4.56	[+2.87, +6.26]	5/5	0.031
CiteSeer	Uniform 0.4	GB-LCSS−random size-matched grouping	+13.16	[+11.26, +15.06]	5/5	¡0.001
CiteSeer	Uniform 0.4	GB-LCSS−MLP+CE	+11.74	[+9.99, +13.50]	5/5	0.001
CiteSeer	Uniform 0.4	GB-LCSS−JoCoR	+5.34	[+2.51, +8.16]	5/5	0.062
CiteSeer	Uniform 0.6	GB-LCSS−random size-matched grouping	+11.66	[+9.60, +13.72]	5/5	0.002
CiteSeer	Uniform 0.6	GB-LCSS−MLP+CE	+8.67	[+4.92, +12.42]	5/5	0.011
CiteSeer	Uniform 0.6	GB-LCSS−JoCoR	+2.95	[+1.09, +4.81]	5/5	0.082
CiteSeer	Pair 0.4	GB-LCSS−random size-matched grouping	+7.78	[+5.04, +10.53]	5/5	0.010
CiteSeer	Pair 0.4	GB-LCSS−MLP+CE	+6.55	[+3.98, +9.13]	5/5	0.011
CiteSeer	Pair 0.4	GB-LCSS−JoCoR	+3.07	[+2.06, +4.07]	5/5	0.020
CiteSeer	Instance 0.4	GB-LCSS−random size-matched grouping	+12.51	[+9.97, +15.05]	5/5	0.003
CiteSeer	Instance 0.4	GB-LCSS−MLP+CE	+11.25	[+8.48, +14.01]	5/5	0.006

Continued on the next page

Table B.1 (continued)

Dataset	Noise	Comparison	Mean difference	95% CI	Positive seeds	Holm-adjusted p
CiteSeer	Instance 0.4	GB-LCSS−JoCoR	+4.72	[+3.63, +5.80]	5/5	0.007
CoraFull	Uniform 0.2	GB-LCSS−random size-matched grouping	+2.44	[+1.48, +3.40]	5/5	0.012
CoraFull	Uniform 0.2	GB-LCSS−MLP+CE	+2.35	[+1.46, +3.25]	5/5	0.011
CoraFull	Uniform 0.2	GB-LCSS−JoCoR	+0.13	[-0.86, +1.12]	3/5	0.852
CoraFull	Uniform 0.4	GB-LCSS−random size-matched grouping	+5.09	[+4.32, +5.87]	5/5	0.001
CoraFull	Uniform 0.4	GB-LCSS−MLP+CE	+5.41	[+4.51, +6.32]	5/5	0.002
CoraFull	Uniform 0.4	GB-LCSS−JoCoR	+1.69	[+0.86, +2.52]	5/5	0.062
CoraFull	Uniform 0.6	GB-LCSS−random size-matched grouping	+6.02	[+5.16, +6.89]	5/5	¡0.001
CoraFull	Uniform 0.6	GB-LCSS−MLP+CE	+6.27	[+5.52, +7.03]	5/5	¡0.001
CoraFull	Uniform 0.6	GB-LCSS−JoCoR	+3.23	[+1.89, +4.57]	5/5	0.042
CoraFull	Pair 0.4	GB-LCSS−random size-matched grouping	+5.98	[+4.75, +7.20]	5/5	0.003
CoraFull	Pair 0.4	GB-LCSS−MLP+CE	+6.32	[+4.66, +7.98]	5/5	0.007
CoraFull	Pair 0.4	GB-LCSS−JoCoR	+3.02	[+2.10, +3.93]	5/5	0.017
CoraFull	Instance 0.4	GB-LCSS−random size-matched grouping	+5.52	[+4.23, +6.80]	5/5	0.004
CoraFull	Instance 0.4	GB-LCSS−MLP+CE	+5.62	[+4.37, +6.86]	5/5	0.004
CoraFull	Instance 0.4	GB-LCSS−JoCoR	+1.78	[+1.22, +2.34]	5/5	0.018
Coauthor-CS	Uniform 0.2	GB-LCSS−random size-matched grouping	+1.28	[+0.70, +1.85]	5/5	0.012
Coauthor-CS	Uniform 0.2	GB-LCSS−MLP+CE	+0.16	[-0.33, +0.65]	3/5	0.418
Coauthor-CS	Uniform 0.2	GB-LCSS−JoCoR	-0.27	[-0.41, -0.13]	0/5	0.062
Coauthor-CS	Uniform 0.4	GB-LCSS−random size-matched grouping	+3.25	[+1.96, +4.55]	5/5	0.012
Coauthor-CS	Uniform 0.4	GB-LCSS−MLP+CE	+2.29	[+1.66, +2.93]	5/5	0.008
Coauthor-CS	Uniform 0.4	GB-LCSS−JoCoR	+1.33	[+1.07, +1.59]	5/5	0.004
Coauthor-CS	Uniform 0.6	GB-LCSS−random size-matched grouping	+8.41	[+5.72, +11.10]	5/5	0.009
Coauthor-CS	Uniform 0.6	GB-LCSS−MLP+CE	+7.16	[+4.58, +9.75]	5/5	0.011
Coauthor-CS	Uniform 0.6	GB-LCSS−JoCoR	+4.61	[+3.37, +5.85]	5/5	0.011
Coauthor-CS	Pair 0.4	GB-LCSS−random size-matched grouping	+10.25	[+8.67, +11.83]	5/5	0.001
Coauthor-CS	Pair 0.4	GB-LCSS−MLP+CE	+10.32	[+8.97, +11.67]	5/5	¡0.001
Coauthor-CS	Pair 0.4	GB-LCSS−JoCoR	+5.22	[+3.82, +6.62]	5/5	0.011
Coauthor-CS	Instance 0.4	GB-LCSS−random size-matched grouping	+3.65	[+2.12, +5.17]	5/5	0.012

Continued on the next page

Table B.1 (continued)

Dataset	Noise	Comparison	Mean difference	95% CI	Positive seeds	Holm-adjusted p
Coauthor-CS	Instance 0.4	GB-LCSS−MLP+CE	+3.07	[+2.15, +4.00]	5/5	0.009
Coauthor-CS	Instance 0.4	GB-LCSS−JoCoR	+1.34	[+0.76, +1.92]	5/5	0.045
Amazon-Photo	Uniform 0.2	GB-LCSS−random size-matched grouping	+5.03	[+3.82, +6.25]	5/5	0.005
Amazon-Photo	Uniform 0.2	GB-LCSS−MLP+CE	+3.72	[+2.63, +4.81]	5/5	0.009
Amazon-Photo	Uniform 0.2	GB-LCSS−JoCoR	+0.46	[-0.99, +1.92]	3/5	0.852
Amazon-Photo	Uniform 0.4	GB-LCSS−random size-matched grouping	+10.66	[+6.55, +14.78]	5/5	0.012
Amazon-Photo	Uniform 0.4	GB-LCSS−MLP+CE	+8.22	[+5.17, +11.26]	5/5	0.011
Amazon-Photo	Uniform 0.4	GB-LCSS−JoCoR	+2.55	[+1.02, +4.07]	5/5	0.078
Amazon-Photo	Uniform 0.6	GB-LCSS−random size-matched grouping	+13.15	[+9.63, +16.67]	5/5	0.006
Amazon-Photo	Uniform 0.6	GB-LCSS−MLP+CE	+12.85	[+8.39, +17.31]	5/5	0.011
Amazon-Photo	Uniform 0.6	GB-LCSS−JoCoR	+4.96	[+0.60, +9.32]	5/5	0.112
Amazon-Photo	Pair 0.4	GB-LCSS−random size-matched grouping	+11.32	[+8.02, +14.63]	5/5	0.008
Amazon-Photo	Pair 0.4	GB-LCSS−MLP+CE	+11.28	[+9.10, +13.46]	5/5	0.003
Amazon-Photo	Pair 0.4	GB-LCSS−JoCoR	+6.95	[+1.90, +12.00]	5/5	0.094
Amazon-Photo	Instance 0.4	GB-LCSS−random size-matched grouping	+12.03	[+8.45, +15.61]	5/5	0.008
Amazon-Photo	Instance 0.4	GB-LCSS−MLP+CE	+10.46	[+6.95, +13.98]	5/5	0.011
Amazon-Photo	Instance 0.4	GB-LCSS−JoCoR	+4.79	[+2.43, +7.15]	5/5	0.062

Appendix C. Sensitivity to noisy validation labels

This experiment examines the effect of validation-label corruption on model selection under uniform 0.4 noise. The noisy-validation protocol keeps the training-noise realization fixed and independently corrupts validation nodes with the same uniform transition mechanism and noise rate, using seed + 1,000,003. Test labels remain unchanged. GB-LCSS and the random size-matched grouping control use the leave-one-out consensus target defined in the main text; the protocol-adapted GSDN and PosteL implementations follow Section 4.3 and Appendix G.

Table C.1. Test accuracy under clean- and noisy-validation protocols (%; mean ± standard deviation over five seeds). Bold indicates the highest mean within each validation-protocol and dataset block.

Validation protocol	Dataset	Method	Test accuracy
Clean validation labels	CiteSeer	GB-LCSS	**55.24±2.04**
Clean validation labels	CiteSeer	Random size-matched grouping	42.08±2.88
Clean validation labels	CiteSeer	GSDN (protocol adaptation)	50.36±3.39
Clean validation labels	CiteSeer	MLP+CE	43.50±3.02
Clean validation labels	CiteSeer	JoCoR	49.91±3.37

Continued on the next page

Table C.1 (continued)

Validation protocol	Dataset	Method	Test accuracy
Clean validation labels	CiteSeer	PosteL (protocol adaptation)	45.84±2.48
Clean validation labels	Cora	GB-LCSS	**57.49±0.76**
Clean validation labels	Cora	Random size-matched grouping	43.31±1.40
Clean validation labels	Cora	GSDN (protocol adaptation)	55.97±4.29
Clean validation labels	Cora	MLP+CE	44.24±2.62
Clean validation labels	Cora	JoCoR	51.72±1.30
Clean validation labels	Cora	PosteL (protocol adaptation)	47.09±1.81
Clean validation labels	CoraFull	GB-LCSS	**37.39±0.51**
Clean validation labels	CoraFull	Random size-matched grouping	32.30±0.85
Clean validation labels	CoraFull	GSDN (protocol adaptation)	35.80±0.66
Clean validation labels	CoraFull	MLP+CE	31.98±0.98
Clean validation labels	CoraFull	JoCoR	35.70±0.32
Clean validation labels	CoraFull	PosteL (protocol adaptation)	36.51±1.01
Noisy validation labels	CiteSeer	GB-LCSS	**54.99±2.18**
Noisy validation labels	CiteSeer	Random size-matched grouping	42.65±2.49
Noisy validation labels	CiteSeer	GSDN (protocol adaptation)	50.60±1.84
Noisy validation labels	CiteSeer	MLP+CE	43.06±2.96
Noisy validation labels	CiteSeer	JoCoR	50.67±1.07
Noisy validation labels	CiteSeer	PosteL (protocol adaptation)	46.27±1.76
Noisy validation labels	Cora	GB-LCSS	**56.81±1.55**
Noisy validation labels	Cora	Random size-matched grouping	43.12±2.07
Noisy validation labels	Cora	GSDN (protocol adaptation)	54.81±4.70
Noisy validation labels	Cora	MLP+CE	43.30±3.65
Noisy validation labels	Cora	JoCoR	50.77±1.98
Noisy validation labels	Cora	PosteL (protocol adaptation)	46.81±1.94
Noisy validation labels	CoraFull	GB-LCSS	**36.90±0.81**
Noisy validation labels	CoraFull	Random size-matched grouping	32.03±0.77
Noisy validation labels	CoraFull	GSDN (protocol adaptation)	36.13±0.69
Noisy validation labels	CoraFull	MLP+CE	31.99±0.85
Noisy validation labels	CoraFull	JoCoR	35.57±1.04
Noisy validation labels	CoraFull	PosteL (protocol adaptation)	35.74±1.14

Under noisy validation labels, GB-LCSS leads random size-matched grouping by 13.69, 12.35, and 4.87 percentage points on Cora, CiteSeer, and CoraFull, respectively, and leads MLP+CE by 13.51, 11.93, and 4.91 points. The Kendall τ between the rankings of the six methods under the two validation protocols is 1.000 on Cora and 0.867 on both CiteSeer and CoraFull, indicating similar method rankings under the two validation protocols.

Appendix D. Additional mechanism analyses

Appendix D.1. Leave-one-out voting and explicit label smoothing

Table D.1. Test accuracy for leave-one-out voting and explicit label-smoothing ablations under uniform 0.4 noise (%; mean ± standard deviation over five seeds). Bold indicates the highest mean for each dataset.

Dataset	Self-vote ablation	GB-LCSS	Random size-matched grouping	Uniform smoothing	Prior smoothing	Leave-one-out gain
Cora	56.28±1.54	**57.49±0.76**	43.31±1.40	44.69±2.90	44.89±2.38	+1.21
CiteSeer	53.52±2.71	**55.24±2.04**	42.08±2.88	43.69±3.31	43.43±2.99	+1.72
CoraFull	37.30±0.57	**37.39±0.51**	32.30±0.85	32.59±1.12	32.63±0.84	+0.09
Coauthor-CS	90.62±0.47	**90.71±0.59**	87.46±1.07	88.25±0.80	88.04±0.73	+0.09
Amazon-Photo	81.33±0.70	**82.12±1.08**	71.45±3.29	74.18±1.72	74.72±1.44	+0.79

The self-vote ablation uses the same tie-handling rule as the main method; its only difference is that the current node's label remains in the local statistics. Across the full noise spectrum, the smallest setting-wise difference between the leave-one-out target and this ablation is −0.49 percentage points. Uniform and prior smoothing modify only the target construction and do not use granular-ball membership.

When the most-voted class is unique, the target mixes the within-ball distribution with the one-hot vector of the most-voted class. When a tie occurs, the empirical distribution is retained. Effective ties occur in 123 of the 125 dataset–noise–seed runs, yielding 911 tied-ball instances that affect 12,788 of 259,000 training-node target instances (4.94%). This treatment makes the target permutation-equivariant under class relabeling.

Appendix D.2. Alternative grouping interfaces

Table D.2 reports the grouping-interface comparison under the leave-one-out consensus target. kNN excludes the query node itself and matches the mean number of training votes per ball for each seed, whereas k-means matches the number of granular balls for that seed. Bold indicates the highest mean for each dataset.

Table D.2. Test accuracy of granular-ball, kNN, and k-means grouping under uniform 0.4 noise (%; mean ± standard deviation over five seeds).

Dataset	Granular ball	kNN	kNN − granular ball	k-means	k-means − granular ball
Cora	**57.49±0.76**	57.21±1.19	−0.27	56.58±1.23	−0.91
CiteSeer	55.24±2.04	56.17±3.24	+0.94	**56.18±1.78**	+0.94
CoraFull	37.39±0.51	**40.50±0.51**	+3.11	39.26±0.48	+1.87
Coauthor-CS	90.71±0.59	90.83±0.57	+0.12	**91.01±0.64**	+0.29
Amazon-Photo	82.12±1.08	**82.23±1.20**	+0.11	81.68±0.66	−0.44

Appendix D.3. Representation-source comparison

Table D.3. Representation-source diagnostic: classification accuracy and number of granular balls.

Dataset	GRACE H	Raw X	A^2X	No. of balls for raw X	No. of balls for A^2X
Cora	57.49±0.76	42.69 (seed 0)	42.69 (seed 0)	1	1
CiteSeer	55.24±2.04	43.08 (seed 0)	43.08 (seed 0)	1	1
CoraFull	37.39±0.51	31.87 (seed 0)	31.87 (seed 0)	1	1

Under the fixed ball-construction rule on these three datasets, neither the raw attributes nor their adjacency-propagated representation yields a usable multi-ball structure, whereas the GRACE representation produces a non-degenerate partition. A further diagnostic with 3 GRACE seeds ×3 downstream seeds gives cross-pretraining-seed standard deviations of 0.13, 0.45, and 0.60 percentage points on Cora, CiteSeer, and CoraFull, respectively; the gain of GB-LCSS over MLP+CE remains positive for every pretraining seed. The mean ARI between partitions obtained from different pretraining seeds is 0.300, with a range of 0.247–0.365.

Appendix E. Additional protocol details

Split sizes are obtained by integer flooring of $0.2N$, $0.1N$, and $0.7N$. When these values do not sum to N, the remaining one or two nodes are left outside the three subsets. The principal comparisons use paired data splits, training-label noise realizations, and downstream seeds.

Appendix F. Full noise spectrum for structure-assisted soft-target methods

Table F.1 reports the five-seed means and sample standard deviations of GB-LCSS and the protocol-adapted implementations of PosteL, ALS, and SALS on the five full-spectrum datasets and five noise settings. All four methods share the same data splits, training-label noise realizations, and graph-free inference protocol.

Table F.1. Full noise-spectrum results for structure-assisted soft-target methods (test accuracy, %; mean ± standard deviation over five seeds). Bold indicates the highest mean for each dataset-noise setting.

Dataset	Noise setting	GB-LCSS	PosteL (protocol adaptation)	ALS (protocol adaptation)	SALS (protocol adaptation)
Cora	Uniform 0.2	**63.25±2.46**	58.36±1.64	55.98±1.42	58.23±1.67
Cora	Uniform 0.4	**57.49±0.76**	47.09±1.81	45.20±1.24	48.27±2.50
Cora	Uniform 0.6	**41.86±2.30**	32.33±2.37	32.72±2.54	35.34±1.74
Cora	Pair 0.4	**51.45±2.80**	44.36±0.65	41.63±3.61	44.16±1.52
Cora	Instance 0.4	**56.84±1.67**	46.62±2.17	45.73±1.34	48.54±1.17
CiteSeer	Uniform 0.2	**63.92±1.12**	58.11±1.11	56.58±2.04	58.36±1.07
CiteSeer	Uniform 0.4	**55.24±2.04**	45.84±2.48	43.77±2.76	47.20±1.69
CiteSeer	Uniform 0.6	**42.02±3.14**	31.88±0.86	32.00±1.57	35.16±2.58
CiteSeer	Pair 0.4	**51.70±2.14**	45.06±1.60	45.72±2.41	45.63±2.70
CiteSeer	Instance 0.4	**55.58±2.54**	46.02±2.54	43.75±2.18	47.51±2.50
CoraFull	Uniform 0.2	43.46±0.57	**44.54±0.31**	41.72±0.22	44.43±0.75
CoraFull	Uniform 0.4	**37.39±0.51**	36.51±1.01	31.91±1.05	37.06±0.78
CoraFull	Uniform 0.6	**27.43±0.89**	23.90±1.16	20.64±0.61	26.10±1.07
CoraFull	Pair 0.4	**34.46±1.23**	32.62±1.71	29.83±0.92	32.85±1.68
CoraFull	Instance 0.4	**36.95±0.53**	35.99±0.51	31.15±1.18	36.58±0.74
Coauthor-CS	Uniform 0.2	**91.54±0.26**	89.01±0.40	91.13±0.57	91.08±0.49
Coauthor-CS	Uniform 0.4	**90.71±0.59**	84.74±1.51	88.55±0.86	88.85±0.82
Coauthor-CS	Uniform 0.6	**87.19±0.69**	74.61±1.95	79.36±1.67	82.99±0.95
Coauthor-CS	Pair 0.4	**86.24±1.18**	78.45±2.28	74.99±2.24	77.85±1.51
Coauthor-CS	Instance 0.4	**90.51±0.43**	85.72±1.35	87.53±1.27	88.33±0.66
Amazon-Photo	Uniform 0.2	**86.07±1.05**	84.37±0.96	82.63±1.27	84.81±1.20
Amazon-Photo	Uniform 0.4	**82.12±1.08**	80.59±0.94	73.41±2.75	77.86±0.79
Amazon-Photo	Uniform 0.6	**71.45±3.75**	64.49±2.99	58.14±1.78	66.29±3.62
Amazon-Photo	Pair 0.4	**73.89±3.73**	67.56±3.57	62.82±6.11	64.83±3.54
Amazon-Photo	Instance 0.4	**79.59±0.88**	77.23±2.77	68.81±2.81	72.73±1.78

Appendix G. Implementation provenance and key parameters

Table G.1. Implementation provenance of the external structure-assisted and distillation methods.

Method	Implementation provenance	Retained core mechanism	Protocol alignment
PosteL	Official-code integration	Posterior-based target construction	Integrated with our MLP, data splits, and noise protocol; iterative pseudo-labeling disabled
ALS	Mechanism adaptation	Adaptive label smoothing, pacing, and KL constraint	Uses our data splits, noise realizations, and MLP student
SALS	Mechanism adaptation	Neighborhood structure-aware label smoothing	Uses our data splits, noise realizations, and MLP student
Graph-MLP	Protocol port	Multi-hop adjacency-based NContrast objective	Uses our noise protocol and a matched MLP classifier
GSDN	Mechanism adaptation	Feature-level and label-level neighborhood self-distillation	Uses our noise protocol and MLP student
GCN(X)+CE teacher	Project implementation based on GLNN response distillation	GCN response distillation	Uses the common teacher and student configurations and our noise protocol
FF-G2M	Common-backbone reimplementation	Low- and high-frequency knowledge distillation	Uses the common GCN teacher, MLP student, and our noise protocol
KRD	Mechanism adaptation	Reliable-knowledge selection based on stability to feature perturbations	Uses the common teacher and student configurations and our noise protocol
VQGraph	Mechanism adaptation	Soft codebook and token-, node-, and edge-level distillation	Uses the common teacher and student configurations and our noise protocol

Table G.2. Method-specific parameters and their sources for the principal comparisons in the main text.

Method	Method-specific settings	Parameter source or selection rule
Small-loss	Forget rate equal to the known synthetic noise rate; linear ramp-up over 10 epochs with exponent 1.0	Unified project implementation; the forget rate is set from the synthetic noise rate
Co-teaching	Two independent MLPs; forget-rate setting identical to Small-loss	Ported from the NoisyGL configuration; the ramp-up schedule is standardized according to the noise rate used here

Continued on the next page

Table G.2 (continued)

Method	Method-specific settings	Parameter source or selection rule
JoCoR	Two independent MLPs; forget-rate setting identical to Small-loss; consistency weight 0.1	Ported from the NoisyGL configuration; the ramp-up schedule is standardized according to the noise rate used here
SCE	Cross-entropy weight 0.1 and reverse-cross-entropy weight 1.0	Ported from the NoisyGL configuration with the original method weights retained
ELR	Exponential moving-average coefficient 0.7 and regularization weight 1.0	Implemented according to the original objective; method-specific parameters were fixed before the full grid
LogitClip	Logit-clipping threshold 1.0	Implemented according to the original objective; method-specific parameters were fixed before the full grid
PosteL (protocol adaptation)	Soft-label ratio 0.8, smoothing ratio 0.4, degree cutoff 1, temperature 1.0; iterative pseudo-labeling disabled	Official no-IPL target construction with the unified project training configuration
ALS (protocol adaptation)	2 propagation steps, residual coefficient 0.5, KL weight 10^{-3}, exponential pacing, and maximum smoothing weight 0.1	Mechanism adaptation using a project-fixed configuration shared across datasets
SALS (protocol adaptation)	$\epsilon = 0.4$, $\gamma = 0.8$; unlabeled neighborhoods fall back to the uniform distribution	Mechanism adaptation using a project-fixed configuration shared across datasets
Graph-MLP (protocol adaptation)	Graph batch size 512, neighborhood-contrastive weight 0.5, and temperature 0.5	Protocol adaptation using a project-fixed configuration with a matched MLP backbone
GSDN (mechanism adaptation)	Graph batch size 512; feature-level and label-level loss weights both 0.8; negative-constraint weight 0.1; initial mixup coefficient 0.5	Mechanism adaptation using a project-fixed configuration shared across datasets
GB-LCSS	$B_{\max} = 64, n_{\min} = 32, \delta = 0.05, s_{\min} = 2, \beta = 0.5$; leave-one-out voting; empirical distribution retained under ties; 2-means uses Euclidean distance, k-means++ initialization, and 10 independent initializations; candidate random state is the downstream seed plus the split iteration and candidate position	Construction parameters were fixed during preliminary development on Cora and CiteSeer and then held unchanged for the full experimental grid; $s_{\min} = 2$ requires at least two other training-node votes
GCN(X)+CE teacher	Two-layer GCN teacher of width 64 and two-layer MLP student of width 64; teacher and student both trained for 200 epochs; temperature 1.0	Common response-distillation configuration based on GLNN [3]
FF-G2M (reimplementation with a common student backbone)	Two-layer GCN teacher of width 64; low- and high-frequency distillation weights 0.4; temperature 1.5	Public core objective retained and fixed under the common teacher and student configurations

Continued on the next page

Table G.2 (continued)

Method	Method-specific settings	Parameter source or selection rule
KRD (mechanism adaptation)	4 feature perturbations with standard deviation 0.01; top 50% of nodes by reliability retained; distillation temperature 1.0	Public core mechanism retained and fixed under the common teacher and student configurations
VQGraph (mechanism adaptation)	128 soft codes; codebook temperature 1.0; token/node/edge loss weights 1.0/0.001/0.03; 4096 edges sampled per epoch	Public core mechanism retained and fixed under the common teacher and student configurations
Granular-ball consensus teacher	Two-layer GCN(X) teacher of width 64 using the leave-one-out consensus target in Eq. (14); student temperature 1.0; student fits only the teacher predictions over all nodes	Proposed distillation extension under the common teacher and student configurations

Parameters labeled as project-fixed were selected before the complete comparison grid and then held constant across datasets. Complete configurations are provided in the supplementary repository.

Table G.3. GRACE configurations used for the five full-spectrum datasets.

Dataset	Configuration source	d_h/d_p	Learning rate	Epochs	Edge-drop rates	Feature-mask rates	Temperature
Cora	Public GRACE configuration	128/128	5×10^{-4}	200	0.2/0.4	0.3/0.4	0.4
CiteSeer	Public GRACE configuration	256/256	10^{-3}	200	0.2/0.0	0.3/0.2	0.9
CoraFull	Fixed transfer of the DBLP configuration	256/256	10^{-3}	1000	0.1/0.4	0.1/0.0	0.7
Coauthor-CS	Fixed transfer of the DBLP configuration	256/256	10^{-3}	1000	0.1/0.4	0.1/0.0	0.7
Amazon-Photo	Fixed transfer of the PubMed configuration	256/256	10^{-3}	1500	0.4/0.1	0.0/0.2	0.7

These pretraining configurations were fixed before the label-noise experiments. GRACE training does not access labels or data-split masks.

Appendix H. Results on additional datasets

The six additional datasets were first evaluated under uniform 0.4 noise. CiteSeerFull was subsequently examined under the other four noise settings as an exploratory follow-up. Table H.1 summarizes the basic statistics of these datasets; data processing and the definition of edge homophily follow Table 1.

Table H.1. Basic statistics of the six additional datasets. Edges are unique undirected pairs after self-loop removal, and the number of training nodes is based on the 20% split.

Dataset	Nodes	Undirected edges	Features	Classes	Training nodes	Edge homophily
PubMed	19,717	44,324	500	3	3,943	0.802
Amazon-Computers	13,752	245,861	767	10	2,750	0.777
Cora_ML	2,995	8,158	2,879	7	599	0.789
Attributed-Wiki	2,405	11,596	4,973	17	481	0.610
CiteSeerFull	4,230	5,337	602	6	846	0.949
Coauthor-Physics	34,493	247,962	8,415	5	6,898	0.931

Table H.2. Uniform 0.4 results on the six additional datasets (test accuracy, %; mean ± standard deviation over five seeds). Bold indicates the highest mean among GB-LCSS, the strongest baseline, and random size-matched grouping for each dataset.

Dataset	GB-LCSS	Strongest baseline	Baseline accuracy	GB-LCSS − baseline	Positive seeds	Random size-matched grouping	GB-LCSS − random size-matched grouping
PubMed	72.60±0.66	JoCoR	**77.84±1.10**	-5.24	0/5	63.04±4.49	+9.56
Amazon-Computers	76.20±0.56	JoCoR	**76.81±0.85**	-0.60	1/5	70.49±1.08	+5.71
Cora_ML	**56.26±2.61**	Small-loss	55.56±3.42	+0.70	3/5	47.81±3.34	+8.45
Attributed-Wiki	65.17±1.42	LogitClip	**67.83±1.51**	-2.66	0/5	59.83±2.13	+5.34
CiteSeerFull	**60.94±1.08**	JoCoR	59.48±1.60	+1.46	4/5	46.69±2.74	+14.25
Coauthor-Physics	**93.13±0.25**	JoCoR	93.09±0.28	+0.04	3/5	88.80±2.12	+4.33

GB-LCSS with the actual granular-ball partition achieves a higher five-seed mean than the random size-matched control on all six datasets, consistent with the interpretation that local membership contributes to soft-target construction. Gains over the strongest noise-robust baseline vary substantially across datasets; on CiteSeerFull, the gain is 1.46 percentage points under uniform 0.4 noise. Table H.3 further reports the full noise spectrum for this dataset.

Table H.3. Full noise-spectrum results on CiteSeerFull (test accuracy, %; mean ± standard deviation over five seeds). Bold indicates the highest mean among GB-LCSS, the strongest baseline, and random size-matched grouping for each noise setting.

Noise setting	GB-LCSS	Strongest baseline	Baseline accuracy	GB-LCSS − baseline	Positive seeds	Random size-matched grouping
Uniform 0.2	**70.71±0.65**	Small-loss	67.47±1.98	+3.24	5/5	58.70±1.88
Uniform 0.4	**60.94±1.08**	JoCoR	59.48±1.60	+1.46	4/5	46.69±2.74
Uniform 0.6	42.45±3.08	JoCoR	**46.65±3.70**	-4.20	0/5	33.13±3.53
Pair 0.4	**55.02±4.07**	JoCoR	51.12±5.88	+3.89	4/5	43.86±3.51
Instance 0.4	**61.49±3.01**	JoCoR	59.92±4.19	+1.57	3/5	47.51±2.21

CiteSeerFull matches or exceeds the strongest baseline in four settings but falls short by 4.20 percentage points under uniform 0.6 noise. Its gains under low- and moderate-rate uniform noise and under pair and instance-dependent noise do not extend to high-rate uniform noise.